

Backend — Database Schema (PostgreSQL 16)

The physical schema for the whole system, written as **PostgreSQL DDL**. It is ORM-agnostic and is the source of truth for migrations. If Prisma is confirmed (T-02), the Prisma schema is derived from this file (see §25).

It implements the entities in [03-domain-model.md](#), the lifecycles in [04-state-machines.md](#) and the rules in the module docs. Schema areas that depend on open decisions are marked **(D-xx)** in comments.

Contents

1. [Conventions](#)
2. [Extensions, schemas and roles](#)
3. [Shared domains and functions](#)
4. [Table index](#)
5. [Enums \(core \)](#)
6. `identity` : [users](#), [sessions](#), [staff](#)
7. `ref` : [reference data](#)
8. `org` : [organizations](#), [KYB](#), [terms](#), [consent](#)
9. `files` : [files and documents](#)
10. `svc` : [requests](#), [quotes](#), [orders](#)
11. `freight`
12. `transport`
13. `warehouse`
14. `customs`
15. `mkt` : [marketplace](#)
16. `fin` : [payments](#), [ledger](#), [invoices](#), [settlements](#)
17. `ops` : [cancellations](#), [cases](#), [insurance](#)
18. `msg` : [conversations and notifications](#)
19. `audit`
20. `sys` [and](#) `cms`
21. [Deferred foreign keys](#)
22. [Triggers, append-only tables and invariants](#)

- 23. [Partitioning and retention](#)
- 24. [Reporting views and grants](#)
- 25. [Prisma mapping notes](#)
- 26. [Seed data](#)

1. Conventions

Topic	Rule
Schemas	One PostgreSQL schema per module area (<code>identity</code> , <code>org</code> , <code>svc</code> , <code>fin</code> ...). A module writes only to its own schema. Cross-schema foreign keys are kept for integrity
Primary keys	<code>id</code> <code>uuid</code> generated by the application as UUIDv7 (time-ordered, index-friendly). Reference tables use stable text codes as keys (e.g. <code>vehicle_types.code</code>)
Human references	<code>reference</code> <code>text</code> <code>UNIQUE</code> with a format <code>CHECK</code> (<code>LX-260148</code> , <code>P0-260083</code> ...). Values come from <code>sys.reference_counters</code> in the same transaction
Timestamps	<code>timestampz</code> everywhere (stored UTC). Every mutable table has <code>created_at</code> and <code>updated_at</code> (set by the trigger in §22). Business dates without time (<code>cargo_ready_date</code>) are <code>date</code> in Riyadh local
Money	<code>core.halalas</code> (<code>bigint</code> ≥ 0 , 1 SAR = 100 halalas). Signed amounts (adjustments) use <code>core.signed_halalas</code> . Currency is SAR only. No <code>numeric</code> / <code>float</code> for money
Rates	<code>core.bps</code> basis points (1500 = 15%, 350 = 3.5%)
Quantities	<code>numeric(12,2)</code> for weights, volumes, pallets and units. <code>integer</code> for marketplace units
Geo	<code>geography(Point, 4326)</code> (PostGIS)
Enums	Native PostgreSQL enums in the <code>core</code> schema. Values are only added (<code>ALTER TYPE ... ADD VALUE</code>), never renamed or removed
Polymorphic links	Exclusive arcs (nullable FK per target + <code>CHECK (num_nonnulls(...) = 1)</code>) where there are ≤ 4 targets. Generic <code>*_type</code> + <code>*_id</code> columns (no FK) for documents, cases, conversations, audit

Topic	Rule
Soft delete	<code>deleted_at</code> only on user-managed catalog data (addresses, products, vehicles, sites, bank accounts). Never on financial or lifecycle rows
Concurrency	<code>version integer NOT NULL DEFAULT 1</code> on aggregates edited concurrently (orders, quotes, products, stock, settlements)
Arrays of file ids	<code>uuid[]</code> for evidence photo lists (no FK possible; the application validates ownership and scan status)
Naming	<code>snake_case</code> , plural tables, <code>ix_</code> indexes, <code>ux_</code> unique indexes, <code>ck_</code> checks, <code>fk_</code> deferred foreign keys
Drafts	Detail tables allow NULLs so multi-step drafts can be saved. Completeness is validated by the application at submit

2. Extensions, schemas and roles

```
CREATE EXTENSION IF NOT EXISTS postgis;      -- geography points, distance queries
CREATE EXTENSION IF NOT EXISTS pg_trgm;      -- fuzzy search (products, ports, cities)
CREATE EXTENSION IF NOT EXISTS citext;       -- case-insensitive emails
CREATE EXTENSION IF NOT EXISTS btree_gist;    -- exclusion constraints (commission rule periods)
CREATE EXTENSION IF NOT EXISTS pgcrypto;      -- digest() for maintenance scripts
-- Optional: pg_partman (monthly partition maintenance), pg_uuidv7 (only if IDs are generated in

CREATE SCHEMA core;      -- enums, domains, shared functions
CREATE SCHEMA identity;  -- users, sessions, staff
CREATE SCHEMA ref;       -- reference data
CREATE SCHEMA org;       -- organizations, KYB, terms, consent
CREATE SCHEMA files;     -- files and documents
CREATE SCHEMA svc;       -- service requests, quotes, orders (shared core)
CREATE SCHEMA freight;   -- shipping execution
CREATE SCHEMA transport; -- fleet, drivers, trips, GPS
CREATE SCHEMA warehouse; -- storage contracts, inventory, releases
CREATE SCHEMA customs;   -- broker authorization, declarations
CREATE SCHEMA mkt;       -- marketplace
CREATE SCHEMA fin;       -- payments, ledger, invoices, settlements, payouts
CREATE SCHEMA ops;       -- cancellations, cases, insurance
CREATE SCHEMA msg;       -- conversations, notifications
CREATE SCHEMA audit;     -- audit log
CREATE SCHEMA sys;       -- outbox, idempotency, flags, approvals, jobs
CREATE SCHEMA cms;       -- FAQs, banners, broadcasts
CREATE SCHEMA reporting; -- masked read-only views (§24)

CREATE ROLE logix_migrator NOLOGIN; -- owns all objects, runs migrations
CREATE ROLE logix_app NOLOGIN;      -- API and workers: SELECT/INSERT/UPDATE/DELETE (append-only)
CREATE ROLE logix_reporting NOLOGIN; -- read replica / Metabase: SELECT on reporting.* only
```

3. Shared domains and functions

```
CREATE DOMAIN core.halalas AS bigint CHECK (VALUE >= 0);
CREATE DOMAIN core.signed_halalas AS bigint;
CREATE DOMAIN core.bps AS integer CHECK (VALUE BETWEEN 0 AND 10000);
CREATE DOMAIN core.currency_code AS char(3) DEFAULT 'SAR' CHECK (VALUE = 'SAR');

-- Keeps updated_at current (attached to every table with updated_at in §22)
CREATE OR REPLACE FUNCTION core.set_updated_at() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END $$;

-- Blocks UPDATE/DELETE on append-only tables (in addition to REVOKE)
CREATE OR REPLACE FUNCTION core.forbid_mutation() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION '% on %.% is not allowed (append-only table)', TG_OP, TG_TABLE_SCHEMA, TG_TABI
END $$;

-- Arabic search normalization: strips diacritics and tatweel, unifies alef/yaa/taa marbuta,
-- converts Arabic-Indic and Persian digits to Western digits, lower-cases Latin text.
CREATE OR REPLACE FUNCTION core.normalize_ar(input text) RETURNS text
LANGUAGE sql IMMUTABLE PARALLEL SAFE AS $$
SELECT lower(
    translate(
        regexp_replace(coalesce(input, ''), '[٠-۹]', '', 'g'),
        '٠١٢٣٤٥٦٧٨٩٠١٢٣٤٥٦٧٨٩اآإةآأءهـو',
        '01234567890123456789ا ا ا ا ا هـ و'
    )
)
$$;
```

4. Table index

Schema	Tables
identity	users , sessions , refresh_tokens , otp_challenges , staff_users , staff_roles , staff_role_permissions , staff_user_roles , staff_sessions , staff_invitations
ref	countries , regions , cities , ports , customs_checkpoints , vehicle_types , container_types , storage_types , handling_services , commodity_categories , units_of_measure , reason_codes , document_types , document_requirements , holidays
org	organizations , memberships , org_workspaces , business_profiles , addresses , provider_activities , service_areas , licenses , bank_accounts , verification_cases , verification_items , terms_documents , consents
files	files , documents , document_versions , document_reviews
svc	service_requests , shipping_request_details , transport_request_details , warehousing_request_details , customs_request_details , request_matches , request_questions , quotes , quote_charges , quote_revisions , orders , order_links , order_milestones , order_events , additional_charges , completion_evidence , receipt_confirmations , ratings
freight	bookings , shipment_references
transport	vehicles , drivers , trip_assignments , trip_events , trip_issues , tracking_points (partitioned)
warehouse	sites , storage_contracts , intake_slots , inventory_items , stock_movements , release_requests , release_request_lines , reconciliations
customs	broker_authorizations , declarations , release_notices
mkt	product_categories , products , product_images , favorites , carts , cart_items , checkout_quotes , purchase_orders , purchase_order_items , shipment_readiness , po_handovers , return_requests , return_request_lines
fin	commission_rules , payment_intents , payment_events , refunds , ledger_accounts , ledger_transactions , ledger_entries , invoice_sequences , invoices , invoice_lines , settlements , settlement_adjustments , payout_batches , payouts

Schema	Tables
ops	cancellation_rules , cancellation_requests , cases , case_internal_notes , insurers , insurance_requests , insurance_policies , insurance_claims
msg	conversations , conversation_participants , messages , content_flags , notifications (partitioned), notification_deliveries , notification_preferences , notification_templates
audit	audit_logs (partitioned)
sys	outbox_events , processed_events , idempotency_keys , reference_counters , feature_flags , app_config , pending_approvals , integration_events , export_jobs , saved_views
cms	faqs , banners , broadcasts

136 tables in total.

5. Enums (core)

Status enums mirror [04-state-machines.md](#) exactly. The transition rules live in the application. The database only guarantees valid values.

```

-- Cross-cutting
CREATE TYPE core.app_locale          AS ENUM ('ar', 'en');
CREATE TYPE core.platform_type       AS ENUM ('ios', 'android', 'web');
CREATE TYPE core.workspace_type      AS ENUM ('CUSTOMER', 'SUPPLIER', 'PROVIDER', 'DRIVER');
CREATE TYPE core.service_type        AS ENUM ('SHIPPING', 'TRANSPORT', 'WAREHOUSING', 'CUSTOMS');
CREATE TYPE core.update_source       AS ENUM ('PROVIDER', 'DRIVER', 'BROKER', 'CUSTOMER', 'SUPPLIER');
CREATE TYPE core.actor_type          AS ENUM ('USER', 'STAFF', 'SYSTEM', 'INTEGRATION');

-- identity
CREATE TYPE core.user_status         AS ENUM ('ACTIVE', 'SUSPENDED', 'DELETED');
CREATE TYPE core.staff_status        AS ENUM ('INVITED', 'ACTIVE', 'DEACTIVATED');
CREATE TYPE core.otp_purpose           AS ENUM ('SIGN_IN', 'CHANGE_PHONE_OLD', 'CHANGE_PHONE_NEW');

-- org
CREATE TYPE core.org_kind            AS ENUM ('INDIVIDUAL', 'BUSINESS');
CREATE TYPE core.org_status          AS ENUM ('ACTIVE', 'SUSPENDED', 'CLOSED');
CREATE TYPE core.membership_role     AS ENUM ('OWNER', 'MANAGER', 'MEMBER', 'DRIVER');
CREATE TYPE core.membership_status   AS ENUM ('INVITED', 'ACTIVE', 'REMOVED');
CREATE TYPE core.workspace_status    AS ENUM ('PENDING_VERIFICATION', 'ACTIVE', 'SUSPENDED');
CREATE TYPE core.provider_activity_type AS ENUM ('FREIGHT_SEA', 'FREIGHT_AIR', 'FREIGHT_LAND',
        'TRANSPORT_CARRIER', 'TRANSPORT_BROKER', 'WAREHOUSE');
CREATE TYPE core.activity_status     AS ENUM ('PENDING', 'APPROVED', 'SUSPENDED', 'REJECTED');
CREATE TYPE core.service_area_type   AS ENUM ('CITY', 'REGION', 'COUNTRY', 'PORT', 'CHECKPOINT');
CREATE TYPE core.license_type        AS ENUM ('COMMERCIAL_REGISTRATION', 'TGA_TRANSPORT_LICENSE',
        'VAT_CERTIFICATE', 'NATIONAL_ADDRESS_PROOF', 'WAREHOUSE_LICENSE', 'OTHER');
CREATE TYPE core.license_status      AS ENUM ('PENDING', 'VALID', 'CHANGES_REQUESTED', 'EXPIRED');
CREATE TYPE core.bank_account_status AS ENUM ('PENDING', 'VERIFIED', 'REJECTED');
CREATE TYPE core.verification_status AS ENUM ('DRAFT', 'SUBMITTED', 'UNDER_REVIEW', 'CHANGES_REQUESTED');
CREATE TYPE core.verification_item_type AS ENUM ('BUSINESS_PROFILE', 'LICENSE', 'BANK_ACCOUNT');
CREATE TYPE core.verification_item_status AS ENUM ('PENDING', 'ACCEPTED', 'CHANGES_REQUESTED');
CREATE TYPE core.terms_audience     AS ENUM ('CUSTOMER', 'SUPPLIER', 'PROVIDER', 'PURCHASE');
CREATE TYPE core.terms_status        AS ENUM ('DRAFT', 'PENDING_APPROVAL', 'PUBLISHED', 'REJECTED');
CREATE TYPE core.consent_key         AS ENUM ('TERMS_ACCEPTANCE', 'PRIVACY_ACCEPTANCE', 'RECEIPT_CONFIRMATION',
        'CANCELLATION_POLICY_ACK', 'RECEIPT_CONFIRMATION', 'INSURANCE_TERMS', 'PURCHASE_TERMS', 'BROKER_TERMS');

-- ref / files
CREATE TYPE core.port_type           AS ENUM ('SEA', 'AIR', 'LAND_BORDER', 'DRY_PORT');
CREATE TYPE core.checkpoint_type     AS ENUM ('SEA', 'AIR', 'LAND', 'DRY_PORT');
CREATE TYPE core.document_reviewer_scope AS ENUM ('STAFF', 'BROKER', 'PROVIDER', 'CUSTOMER', 'NATIONALITY');
CREATE TYPE core.requirement_level   AS ENUM ('REQUIRED', 'AS_APPLICABLE', 'OPTIONAL');
CREATE TYPE core.file_purpose         AS ENUM ('DOCUMENT', 'EVIDENCE_PHOTO', 'SIGNATURE', 'PICTURE');

```

```

        'AVATAR', 'EXPORT', 'GENERATED_PDF'));
CREATE TYPE core.scan_status AS ENUM ('PENDING', 'CLEAN', 'INFECTED', 'FAILED');
CREATE TYPE core.document_owner_type AS ENUM ('ORGANIZATION', 'SERVICE_REQUEST', 'ORDER', 'INSURANCE_REQUEST', 'INSURANCE_CLAIM', 'VEHICLE');
CREATE TYPE core.document_status AS ENUM ('MISSING', 'UPLOADED', 'UNDER_REVIEW', 'ACCEPTED', 'EXPIRED', 'REJECTED');
CREATE TYPE core.document_visibility AS ENUM ('OWNER', 'ASSIGNED_PROVIDER', 'ALL_PARTIES');
CREATE TYPE core.review_decision AS ENUM ('ACCEPTED', 'CHANGES_REQUESTED', 'REJECTED');

-- SVC
CREATE TYPE core.request_status AS ENUM ('DRAFT', 'SUBMITTED', 'QUOTED', 'AWAITING_PAYMENT', 'AWAITING_ACCEPTANCE', 'DISPUTED', 'COMPLETED');
CREATE TYPE core.quote_status AS ENUM ('SUBMITTED', 'WITHDRAWN', 'EXPIRED', 'ACCEPTED');
CREATE TYPE core.order_status AS ENUM ('PENDING_PAYMENT', 'VOID', 'CONFIRMED', 'SCHEDULED', 'ON_HOLD', 'IN_BATCH', 'PAID', 'COMPLETED');
CREATE TYPE core.payment_summary_status AS ENUM ('UNPAID', 'PAID', 'PARTIALLY_REFUNDED', 'REFUNDED');
CREATE TYPE core.settlement_status AS ENUM ('SCHEDULED', 'ON_HOLD', 'IN_BATCH', 'PAID', 'COMPLETED');
CREATE TYPE core.milestone_status AS ENUM ('PENDING', 'DONE', 'SKIPPED');
CREATE TYPE core.order_link_type AS ENUM ('CUSTOMS_FOR_SHIPMENT', 'DELIVERY_LEG', 'TRANSPORT', 'PURCHASE_DELIVERY', 'RETURN_PICKUP', 'RELEASE');
CREATE TYPE core.additional_charge_status AS ENUM ('PROPOSED', 'APPROVED', 'REJECTED', 'PAID');
CREATE TYPE core.receipt_condition AS ENUM ('INTACT', 'SHORTAGE', 'DAMAGE');
CREATE TYPE core.rating_status AS ENUM ('PUBLISHED', 'HIDDEN');
CREATE TYPE core.shipping_mode AS ENUM ('SEA', 'AIR', 'LAND');
CREATE TYPE core.trade_direction AS ENUM ('IMPORT', 'EXPORT');
CREATE TYPE core.container_size AS ENUM ('FT20', 'FT40', 'LCL');
CREATE TYPE core.transport_scope AS ENUM ('DOMESTIC', 'CROSS_BORDER');
CREATE TYPE core.loading_assistance AS ENUM ('ASSISTANCE', 'FORKLIFT', 'NONE');
CREATE TYPE core.weight_unit AS ENUM ('KG', 'TON');
CREATE TYPE core.space_unit AS ENUM ('PALLET_POSITIONS', 'SQM');
CREATE TYPE core.warehouse_transport AS ENUM ('LINKED', 'SELF');
CREATE TYPE core.customs_movement AS ENUM ('IMPORT', 'EXPORT', 'TRANSIT');

-- freight / transport
CREATE TYPE core.carrier_type AS ENUM ('SHIPPING_LINE', 'AIRLINE', 'ROAD_CARRIER', 'TRUCK');
CREATE TYPE core.vehicle_status AS ENUM ('ACTIVE', 'INACTIVE', 'SUSPENDED');
CREATE TYPE core.driver_status AS ENUM ('INVITED', 'ACTIVE', 'SUSPENDED');
CREATE TYPE core.assignment_status AS ENUM ('ACTIVE', 'REPLACED', 'CANCELLED', 'COMPLETED');
CREATE TYPE core.trip_event_type AS ENUM ('START_TO_PICKUP', 'ARRIVED_AT_PICKUP', 'CARGO_LOADED', 'BORDER_CROSSED', 'ARRIVED_AT_DROPOFF', 'CORRIDOR_CLOSURE');
CREATE TYPE core.trip_issue_type AS ENUM ('DELAY', 'ACCESS', 'BREAKDOWN', 'OTHER');

-- warehouse / customs
CREATE TYPE core.storage_contract_status AS ENUM ('PENDING_INTAKE', 'ACTIVE', 'CLOSING', 'RECONCILING');

```

```

CREATE TYPE core.intake_slot_status      AS ENUM ('PROPOSED', 'CONFIRMED', 'RECEIVED', 'MISSED');
CREATE TYPE core.stock_movement_type     AS ENUM ('INTAKE', 'RELEASE', 'ADJUSTMENT', 'RETURN');
CREATE TYPE core.release_status           AS ENUM ('REQUESTED', 'APPROVED', 'REJECTED', 'PICKING
                                                'HANDED_OVER', 'COMPLETED', 'CANCELLED');

CREATE TYPE core.collection_method        AS ENUM ('LINKED_CARRIER', 'SELF');
CREATE TYPE core.reconciliation_status    AS ENUM ('DRAFT', 'SUBMITTED', 'ACCEPTED', 'DISPUTED');
CREATE TYPE core.broker_authorization_status AS ENUM ('REQUESTED', 'PENDING_EXTERNAL_ACTION', 'I
CREATE TYPE core.declaration_status        AS ENUM ('NOT_STARTED', 'SUBMITTED', 'UNDER_INSPECTION
                                                'RELEASED', 'HELD');

CREATE TYPE core.duties_payer              AS ENUM ('CUSTOMER_DIRECT', 'BROKER_ON_BEHALF');
CREATE TYPE core.port_collection           AS ENUM ('LINK_TRANSPORT', 'SELECT_CARRIER', 'SELF');

-- mkt
CREATE TYPE core.product_status            AS ENUM ('DRAFT', 'PENDING_REVIEW', 'PUBLISHED', 'PAUSI
CREATE TYPE core.product_condition         AS ENUM ('NEW');
CREATE TYPE core.vat_treatment             AS ENUM ('INCLUSIVE', 'EXCLUSIVE');
CREATE TYPE core.shipping_method           AS ENUM ('LOGIX_TRANSPORT', 'SUPPLIER_DELIVERY', 'PICKI
CREATE TYPE core.po_status                 AS ENUM ('PENDING_PAYMENT', 'VOID', 'PLACED', 'REJECTEI
                                                'READY_FOR_PICKUP', 'HANDED_TO_CARRIER', 'DEL:
                                                'RECEIVED_WITH_ISSUE', 'COMPLETED', 'CANCELLA

CREATE TYPE core.return_reason             AS ENUM ('DEFECT', 'PRE_DELIVERY_DAMAGE', 'MISMATCH');
CREATE TYPE core.return_preference         AS ENUM ('REPLACEMENT', 'REFUND');
CREATE TYPE core.return_status             AS ENUM ('REQUESTED', 'SUPPLIER_REVIEW', 'INFO_REQUESTI
                                                'PICKUP_SCHEDULED', 'RETURNED', 'RESOLVED', '(

CREATE TYPE core.return_resolution         AS ENUM ('REPLACEMENT_SENT', 'REFUNDED', 'PARTIAL_REFUI
CREATE TYPE core.cost_bearer              AS ENUM ('SELLER', 'BUYER', 'PLATFORM', 'PROVIDER');

-- fin
CREATE TYPE core.payable_type              AS ENUM ('ORDER', 'PURCHASE_ORDER', 'ADDITIONAL_CHARGE
CREATE TYPE core.payment_method            AS ENUM ('MADA', 'CARD', 'APPLE_PAY', 'STC_PAY', 'BUSII
CREATE TYPE core.payment_intent_status     AS ENUM ('INITIATED', 'PENDING', 'PAID', 'FAILED', 'EXI
                                                'PARTIALLY_REFUNDED', 'REFUNDED');

CREATE TYPE core.reconciliation_state      AS ENUM ('PENDING', 'MATCHED', 'MISMATCH', 'INVESTIGATI
CREATE TYPE core.refund_status             AS ENUM ('REQUESTED', 'PROCESSING', 'SUCCEEDED', 'FAILI
CREATE TYPE core.refund_reason             AS ENUM ('CANCELLATION', 'RETURN', 'DISPUTE', 'SUPPLIEI
CREATE TYPE core.ledger_account_type       AS ENUM ('ASSET', 'LIABILITY', 'REVENUE', 'EXPENSE', 'I
CREATE TYPE core.entry_direction           AS ENUM ('DEBIT', 'CREDIT');
CREATE TYPE core.commission_category       AS ENUM ('FREIGHT_TRANSPORT', 'CUSTOMS_STORAGE', 'MARKI
CREATE TYPE core.invoice_type              AS ENUM ('SERVICE_INVOICE', 'COMMISSION_INVOICE', 'PLA
CREATE TYPE core.zatca_status              AS ENUM ('NOT_REQUIRED', 'PENDING', 'CLEARED', 'REPORTI
CREATE TYPE core.beneficiary_role          AS ENUM ('PROVIDER', 'SUPPLIER');
CREATE TYPE core.adjustment_type           AS ENUM ('REFUND_DEDUCTION', 'DAMAGE_DEDUCTION', 'REtui
                                                'CANCELLATION_FEE_COMPENSATION', 'MANUAL');

```

```

CREATE TYPE core.payout_batch_status AS ENUM ('DRAFT', 'PENDING_APPROVAL', 'APPROVED', 'EXP
CREATE TYPE core.payout_status AS ENUM ('PENDING', 'EXPORTED', 'PAID', 'FAILED');

-- ops
CREATE TYPE core.cancellation_outcome AS ENUM ('FEE_PERCENT', 'REQUIRES_REVIEW', 'NOT_ALLOWEI
CREATE TYPE core.cancellation_status AS ENUM ('REQUESTED', 'AUTO_APPROVED', 'UNDER_REVIEW',
        'REFUND_PROCESSING', 'COMPLETED');

CREATE TYPE core.cancellation_requester AS ENUM ('CUSTOMER', 'PROVIDER', 'SUPPLIER', 'ADMIN');
CREATE TYPE core.case_type AS ENUM ('DAMAGE_SHORTAGE', 'CANCELLATION_REVIEW', 'RE
        'DOCUMENT_ISSUE', 'SERVICE_ISSUE', 'INSURANCE_
CREATE TYPE core.case_status AS ENUM ('OPEN', 'IN_PROGRESS', 'WAITING_ON_CUSTOMER',
        'WAITING_ON_SUPPLIER', 'RESOLVED', 'REOPENED'.
CREATE TYPE core.case_priority AS ENUM ('LOW', 'NORMAL', 'HIGH', 'URGENT');
CREATE TYPE core.case_subject_type AS ENUM ('ORDER', 'PURCHASE_ORDER', 'RETURN', 'PAYMENT
CREATE TYPE core.case_resolution_code AS ENUM ('RESOLVED_REFUND', 'RESOLVED_ADJUSTMENT', 'RE
        'RESOLVED_INSURANCE', 'DUPLICATE', 'INVALID',
CREATE TYPE core.insurance_request_status AS ENUM ('REQUESTED', 'UNDER_INSURER_REVIEW', 'QUOTED
        'DECLINED', 'EXPIRED', 'CANCELLED');

CREATE TYPE core.insurance_payment_route AS ENUM ('IN_APP', 'DIRECT_TO_INSURER');
CREATE TYPE core.insurance_claim_status AS ENUM ('SUBMITTED', 'WITH_INSURER', 'APPROVED', 'REJI

-- msg / sys / cms
CREATE TYPE core.conversation_type AS ENUM ('ORDER', 'REQUEST_CLARIFICATION', 'PRODUCT_INC
        'CASE', 'SUPPORT', 'RETURN');

CREATE TYPE core.conversation_status AS ENUM ('OPEN', 'CLOSED', 'LOCKED');
CREATE TYPE core.participant_role AS ENUM ('CUSTOMER', 'PROVIDER', 'SUPPLIER', 'DRIVER',
CREATE TYPE core.message_kind AS ENUM ('TEXT', 'ATTACHMENT', 'SYSTEM');
CREATE TYPE core.flag_status AS ENUM ('OPEN', 'DISMISSED', 'WARNED', 'ESCALATED');
CREATE TYPE core.notification_channel AS ENUM ('IN_APP', 'PUSH', 'SMS', 'EMAIL');
CREATE TYPE core.notification_category AS ENUM ('ORDERS', 'QUOTES', 'MESSAGES', 'FINANCE', 'AC
CREATE TYPE core.notification_priority AS ENUM ('LOW', 'NORMAL', 'HIGH');
CREATE TYPE core.delivery_status AS ENUM ('QUEUED', 'SENT', 'DELIVERED', 'FAILED', 'SKII
CREATE TYPE core.approval_status AS ENUM ('PENDING', 'APPROVED', 'REJECTED', 'EXPIRED');
CREATE TYPE core.export_status AS ENUM ('QUEUED', 'RUNNING', 'SUCCEEDED', 'FAILED', 'I
CREATE TYPE core.broadcast_status AS ENUM ('DRAFT', 'PENDING_APPROVAL', 'SCHEDULED', 'SEI

```

6. identity : users, sessions, staff

```
CREATE TABLE identity.users (  
  id                uuid PRIMARY KEY,  
  phone_e164        text NOT NULL,           -- Saudi-only at launch is enforced in 1  
  email             citext,  
  full_name         text,  
  locale            core.app_locale NOT NULL DEFAULT 'ar',  
  status            core.user_status NOT NULL DEFAULT 'ACTIVE',  
  last_login_at     timestamptz,  
  deletion_requested_at timestamptz,  
  anonymized_at     timestamptz,           -- PDPL deletion: personal fields replac  
  created_at        timestamptz NOT NULL DEFAULT now(),  
  updated_at        timestamptz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_users_phone CHECK (phone_e164 ~ '^\\+[1-9][0-9]{7,14}$'),  
  CONSTRAINT ck_users_name CHECK (full_name IS NULL OR char_length(full_name) BETWEEN 2 AND 80);  
  
CREATE UNIQUE INDEX ux_users_phone ON identity.users (phone_e164);  
CREATE INDEX ix_users_email ON identity.users (email) WHERE email IS NOT NULL;  
  
CREATE TABLE identity.sessions (  
  id                uuid PRIMARY KEY,  
  user_id           uuid NOT NULL REFERENCES identity.users(id),  
  device_id         text NOT NULL,  
  platform          core.platform_type NOT NULL,  
  app_version       text,  
  push_token        text,  
  push_token_updated_at timestamptz,  
  ip                inet,  
  user_agent        text,  
  last_seen_at      timestamptz NOT NULL DEFAULT now(),  
  expires_at        timestamptz NOT NULL,           -- refresh lifetime (30 days, sliding)  
  revoked_at        timestamptz,  
  revoke_reason     text,           -- LOGOUT | REMOTE_SIGN_OUT | TOKEN_REUSE  
  created_at        timestamptz NOT NULL DEFAULT now(),  
  updated_at        timestamptz NOT NULL DEFAULT now()  
);  
  
CREATE INDEX ix_sessions_user_active ON identity.sessions (user_id) WHERE revoked_at IS NULL;  
CREATE UNIQUE INDEX ux_sessions_push_token ON identity.sessions (push_token)  
  WHERE push_token IS NOT NULL AND revoked_at IS NULL;
```

-- Rotating refresh tokens. Presenting a token that was already rotated revokes the whole session

```

CREATE TABLE identity.refresh_tokens (
  id          uuid PRIMARY KEY,
  session_id  uuid NOT NULL REFERENCES identity.sessions(id),
  token_hash  bytea NOT NULL,
  issued_at   timestampz NOT NULL DEFAULT now(),
  rotated_at  timestampz,
  revoked_at  timestampz
);

CREATE UNIQUE INDEX ux_refresh_tokens_hash ON identity.refresh_tokens (token_hash);
CREATE INDEX ix_refresh_tokens_session ON identity.refresh_tokens (session_id);

-- Live OTP state is in Redis. This table is the audit trail and fallback (retention 30 days).
CREATE TABLE identity.otp_challenges (
  id          uuid PRIMARY KEY,
  phone_e164   text NOT NULL,
  purpose      core.otp_purpose NOT NULL,
  code_hash    bytea NOT NULL,          -- HMAC-SHA256(pepper, code); the code :
  attempts     smallint NOT NULL DEFAULT 0,
  max_attempts  smallint NOT NULL DEFAULT 5,
  send_count    smallint NOT NULL DEFAULT 1,
  resend_available_at timestampz NOT NULL,    -- +45 s
  expires_at    timestampz NOT NULL,        -- +5 min
  verified_at   timestampz,
  locked_at     timestampz,
  user_id       uuid REFERENCES identity.users(id),
  ip            inet,
  device_id     text,
  sms_provider   text,
  sms_message_id text,
  created_at     timestampz NOT NULL DEFAULT now(),
  updated_at     timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_otp_attempts CHECK (attempts <= max_attempts)
);

CREATE INDEX ix_otp_phone_created ON identity.otp_challenges (phone_e164, created_at DESC);

CREATE TABLE identity.staff_users (
  id          uuid PRIMARY KEY,
  email        citext NOT NULL UNIQUE,
  full_name    text NOT NULL,
  password_hash text,                  -- argon2id; NULL until the invitation :
  totp_secret_enc bytea,              -- encrypted with a KMS envelope key
  totp_enrolled_at timestampz,
  backup_code_hashes text[] NOT NULL DEFAULT '{}',

```

```

status          core.staff_status NOT NULL DEFAULT 'INVITED',
locale          core.app_locale NOT NULL DEFAULT 'ar',
ip_allowlist    cidr[], -- required for finance / super admin (I
failed_login_count smallint NOT NULL DEFAULT 0,
locked_until    timestamptz,
last_login_at   timestamptz,
linked_user_id  uuid REFERENCES identity.users(id), -- conflict-of-interest checks when si
created_at      timestamptz NOT NULL DEFAULT now(),
updated_at      timestamptz NOT NULL DEFAULT now(),
CONSTRAINT ck_staff_active_2fa CHECK (status <> 'ACTIVE' OR (password_hash IS NOT NULL AND to
);

```

```

CREATE TABLE identity.staff_roles (
  id          uuid PRIMARY KEY,
  code        text NOT NULL UNIQUE, -- SUPER_ADMIN, OPS_AGENT, KYB_REVIEWER.
  name_ar     text NOT NULL,
  name_en     text NOT NULL,
  is_builtin  boolean NOT NULL DEFAULT false, -- built-in roles are locked (clone to c
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE identity.staff_role_permissions (
  role_id      uuid NOT NULL REFERENCES identity.staff_roles(id) ON DELETE CASCADE,
  permission   text NOT NULL, -- e.g. 'finance.payouts.approve'
  PRIMARY KEY (role_id, permission),
  CONSTRAINT ck_permission_format CHECK (permission ~ '^[a-z_]+(\.[a-z_]+){1,3}$')
);

```

```

CREATE TABLE identity.staff_user_roles (
  staff_user_id uuid NOT NULL REFERENCES identity.staff_users(id) ON DELETE CASCADE,
  role_id        uuid NOT NULL REFERENCES identity.staff_roles(id),
  granted_by     uuid REFERENCES identity.staff_users(id),
  granted_at     timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (staff_user_id, role_id)
);

```

```

CREATE TABLE identity.staff_sessions (
  id          uuid PRIMARY KEY,
  staff_user_id uuid NOT NULL REFERENCES identity.staff_users(id),
  refresh_token_hash bytea NOT NULL UNIQUE,
  ip          inet,
  user_agent  text,

```

```

mfa_verified_at    timestampz NOT NULL,
last_seen_at       timestampz NOT NULL DEFAULT now(),
idle_expires_at    timestampz NOT NULL,           -- 30 min idle timeout, extended on act:
absolute_expires_at timestampz NOT NULL,           -- login + 8 h
revoked_at         timestampz,
created_at         timestampz NOT NULL DEFAULT now(),
updated_at         timestampz NOT NULL DEFAULT now()
);

CREATE INDEX ix_staff_sessions_user ON identity.staff_sessions (staff_user_id) WHERE revoked_at

CREATE TABLE identity.staff_invitations (
  id          uuid PRIMARY KEY,
  email       citext NOT NULL,
  role_ids    uuid[] NOT NULL,
  token_hash  bytea NOT NULL UNIQUE,
  invited_by  uuid NOT NULL REFERENCES identity.staff_users(id),
  expires_at  timestampz NOT NULL,                 -- 48 h, single use
  accepted_at timestampz,
  revoked_at  timestampz,
  created_at  timestampz NOT NULL DEFAULT now()
);

```

7. ref : reference data

All catalogs are bilingual, admin-managed and never hard-deleted once referenced (is_active = false instead).

```

CREATE TABLE ref.countries (
  code          char(2) PRIMARY KEY,                -- ISO 3166-1 alpha-2
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  phone_code    text,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE ref.regions (
  code          text PRIMARY KEY,                    -- e.g. 'SA-01'
  country_code  char(2) NOT NULL REFERENCES ref.countries(code),
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE ref.cities (
  id            uuid PRIMARY KEY,
  code          text NOT NULL UNIQUE,                -- e.g. 'SA-RUH', 'AE-DXB'
  country_code  char(2) NOT NULL REFERENCES ref.countries(code),
  region_code   text REFERENCES ref.regions(code),
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  location      geography(Point, 4326),
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

```

```

CREATE INDEX ix_cities_country ON ref.cities (country_code) WHERE is_active;

```

```

CREATE INDEX ix_cities_search ON ref.cities USING gin (core.normalize_ar(name_ar || ' ' || name_en));

```

```

CREATE TABLE ref.ports (
  id            uuid PRIMARY KEY,
  code          text NOT NULL UNIQUE,                -- internal code (also covers land borders)
  port_type     core.port_type NOT NULL,
  unlocode      char(5),                            -- e.g. CNSHA, SAJED

```

```

iata_code      char(3),                                -- airports, e.g. RUH, DXB
country_code   char(2) NOT NULL REFERENCES ref.countries(code),
city_id        uuid REFERENCES ref.cities(id),
name_ar        text NOT NULL,
name_en        text NOT NULL,
location       geography(Point, 4326),
is_active      boolean NOT NULL DEFAULT true,
created_at     timestampz NOT NULL DEFAULT now(),
updated_at     timestampz NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX ux_ports_unlocode ON ref.ports (unlocode) WHERE unlocode IS NOT NULL;
CREATE UNIQUE INDEX ux_ports_iata ON ref.ports (iata_code) WHERE iata_code IS NOT NULL;
CREATE INDEX ix_ports_type_country ON ref.ports (port_type, country_code) WHERE is_active;
CREATE INDEX ix_ports_search ON ref.ports USING gin (
    core.normalize_ar(name_ar || ' ' || name_en || ' ' || coalesce(unlocode, '')) || ' ' || coalesce(

CREATE TABLE ref.customs_checkpoints (
    id            uuid PRIMARY KEY,
    code          text NOT NULL UNIQUE,
    checkpoint_type core.checkpoint_type NOT NULL,
    port_id       uuid REFERENCES ref.ports(id),
    name_ar       text NOT NULL,
    name_en       text NOT NULL,
    supported_movements core.customs_movement[] NOT NULL DEFAULT '{IMPORT,EXPORT,TRANSIT}',
    is_active      boolean NOT NULL DEFAULT true,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now()
);

CREATE TABLE ref.vehicle_types (
    code          text PRIMARY KEY,                                -- DRY_TRUCK, REEFER, LIGHT_TRUCK, LOWBI
    name_ar       text NOT NULL,
    name_en       text NOT NULL,
    capacity_ton   numeric(6,2),
    is_reefer      boolean NOT NULL DEFAULT false,
    is_car_carrier boolean NOT NULL DEFAULT false,
    is_light       boolean NOT NULL DEFAULT false,
    icon          text,                                             -- handoff icon name
    is_active      boolean NOT NULL DEFAULT true,
    sort_order     integer NOT NULL DEFAULT 0,
    created_at     timestampz NOT NULL DEFAULT now(),
    updated_at     timestampz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE ref.container_types (
  code          text PRIMARY KEY,                -- 20DC, 40DC, 40HC, 20RF, 40RF, 20OT, '
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  size_ft       smallint CHECK (size_ft IN (20, 40, 45)),
  is_reefer     boolean NOT NULL DEFAULT false,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE ref.storage_types (
  code          text PRIMARY KEY,                -- DRY, CHILLED, FROZEN, OTHER
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  requires_temperature boolean NOT NULL DEFAULT false,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE ref.handling_services (
  code          text PRIMARY KEY,                -- RECEIVING, LOADING, UNLOADING, SORTING
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE ref.units_of_measure (
  code          text PRIMARY KEY,                -- PIECE, BOX, CARTON, PALLET, KG, TON,
  kind          text NOT NULL CHECK (kind IN ('COUNT', 'WEIGHT', 'VOLUME', 'AREA', 'CONTAINER')),
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now()
);

```

```

);

CREATE TABLE ref.commodity_categories (
  code          text PRIMARY KEY,
  parent_code   text REFERENCES ref.commodity_categories(code),
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  saber_hint    boolean NOT NULL DEFAULT false,          -- show "SABER certificate – as applical
  is_hazardous  boolean NOT NULL DEFAULT false,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now()
);

-- One table for all reason pickers and rejection templates
CREATE TABLE ref.reason_codes (
  id            uuid PRIMARY KEY,
  domain        text NOT NULL CHECK (domain IN ('CANCELLATION', 'RETURN', 'TRIP_ISSUE', 'CASI
              'PRODUCT_MODERATION', 'REQUEST_DECLINE', 'SU

  code          text NOT NULL,
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  user_message_ar text,                                -- shown to app users, e.g. "A clearer c
  user_message_en text,
  is_active     boolean NOT NULL DEFAULT true,
  sort_order    integer NOT NULL DEFAULT 0,
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now(),
  UNIQUE (domain, code)
);

CREATE TABLE ref.document_types (
  code          text PRIMARY KEY,                                -- COMMERCIAL_REGISTRATION, BILL_OF_LAD:
  name_ar       text NOT NULL,
  name_en       text NOT NULL,
  help_ar       text,
  help_en       text,
  reviewer_scope core.document_reviewer_scope NOT NULL,
  has_expiry    boolean NOT NULL DEFAULT false,
  allowed_mime  text[] NOT NULL DEFAULT '{application/pdf,image/jpeg,image/png}',
  max_size_mb   smallint NOT NULL DEFAULT 10 CHECK (max_size_mb BETWEEN 1 AND 50),
  is_active     boolean NOT NULL DEFAULT true,

```

```

    sort_order      integer NOT NULL DEFAULT 0,
    created_at       timestampz NOT NULL DEFAULT now(),
    updated_at       timestampz NOT NULL DEFAULT now()
);

```

-- Checklist rules: which documents are required, per KYB activity or per service / mode / mover

```

CREATE TABLE ref.document_requirements (
    id                uuid PRIMARY KEY,
    context            text NOT NULL CHECK (context IN ('KYB', 'SERVICE_REQUEST', 'ORDER', 'FLEE'
workspace            core.workspace_type,
activity             core.provider_activity_type,
service_type         core.service_type,
shipping_mode        core.shipping_mode,
customs_movement     core.customs_movement,
document_type_code   text NOT NULL REFERENCES ref.document_types(code),
requirement          core.requirement_level NOT NULL,
stage                text,                                -- AT_SUBMIT | BEFORE_DECLARATION | BEFO
effective_from       timestampz NOT NULL DEFAULT now(),
effective_to         timestampz,
is_active            boolean NOT NULL DEFAULT true,
created_at           timestampz NOT NULL DEFAULT now(),
updated_at           timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_doc_req_period CHECK (effective_to IS NULL OR effective_to > effective_from)
);

CREATE INDEX ix_doc_requirements_lookup ON ref.document_requirements (context, service_type, cu
WHERE is_active;

```

-- Business-day calendar (weekend days live in sys.app_config 'calendar.weekend')

```

CREATE TABLE ref.holidays (
    holiday_date      date PRIMARY KEY,
    name_ar           text NOT NULL,
    name_en           text NOT NULL,
    created_by_staff_id uuid REFERENCES identity.staff_users(id),
    created_at         timestampz NOT NULL DEFAULT now(),
    updated_at         timestampz NOT NULL DEFAULT now()
);

```

8. org : organizations, KYB, terms, consent

-- The account that owns everything. Individual customers get a one-member INDIVIDUAL organization:

```
CREATE TABLE org.organizations (  
  id                uuid PRIMARY KEY,  
  kind              core.org_kind NOT NULL,  
  display_name      text NOT NULL,  
  status            core.org_status NOT NULL DEFAULT 'ACTIVE',  
  suspension_reason text,  
  default_locale    core.app_locale NOT NULL DEFAULT 'ar',  
  rating_avg        numeric(3,2) CHECK (rating_avg BETWEEN 1 AND 5),  
  rating_count      integer NOT NULL DEFAULT 0,  
  completed_jobs_count integer NOT NULL DEFAULT 0,      -- "86 trips" on quote cards  
  reliability_score  numeric(5,2),                    -- internal (cancellations, lateness)  
  created_by_user_id uuid REFERENCES identity.users(id),  
  version           integer NOT NULL DEFAULT 1,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  updated_at        timestampz NOT NULL DEFAULT now()  
);  
  
CREATE INDEX ix_organizations_name ON org.organizations USING gin (core.normalize_ar(display_name));
```

```
CREATE TABLE org.memberships (  
  id                uuid PRIMARY KEY,  
  user_id           uuid NOT NULL REFERENCES identity.users(id),  
  organization_id   uuid NOT NULL REFERENCES org.organizations(id),  
  role              core.membership_role NOT NULL,  
  status            core.membership_status NOT NULL DEFAULT 'ACTIVE',  
  invited_by_user_id uuid REFERENCES identity.users(id),  
  joined_at         timestampz,  
  removed_at        timestampz,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  updated_at        timestampz NOT NULL DEFAULT now(),  
  UNIQUE (user_id, organization_id)  
);  
  
CREATE INDEX ix_memberships_org ON org.memberships (organization_id, role) WHERE status = 'ACTIVE';  
-- MVP: exactly one active owner per organization (D-19)  
CREATE UNIQUE INDEX ux_memberships_one_owner ON org.memberships (organization_id)  
  WHERE role = 'OWNER' AND status = 'ACTIVE';
```

```
CREATE TABLE org.org_workspaces (  
  id                uuid PRIMARY KEY,  
  organization_id   uuid NOT NULL REFERENCES org.organizations(id),
```

```

workspace          core.workspace_type NOT NULL,
status             core.workspace_status NOT NULL DEFAULT 'PENDING_VERIFICATION',
activated_at       timestampz,
suspended_at       timestampz,
suspension_reason  text,
created_at         timestampz NOT NULL DEFAULT now(),
updated_at         timestampz NOT NULL DEFAULT now(),
UNIQUE (organization_id, workspace),
CONSTRAINT ck_org_workspace_kind CHECK (workspace IN ('CUSTOMER', 'SUPPLIER', 'PROVIDER')) --
);

```

```

CREATE TABLE org.addresses (
  id                uuid PRIMARY KEY,
  organization_id   uuid NOT NULL REFERENCES org.organizations(id),
  label             text,                                -- "Main warehouse", "Gate 3"
  contact_name      text,
  contact_phone     text,
  country_code      char(2) NOT NULL DEFAULT 'SA' REFERENCES ref.countries(code),
  city_id           uuid REFERENCES ref.cities(id),
  district          text,
  street            text,
  building_number   text,
  postal_code       text,
  additional_number text,
  short_address     text,                                -- Saudi national short address, e.g. RI
  address_line      text,                                -- free text for non-Saudi addresses
  location          geography(Point, 4326),
  is_default        boolean NOT NULL DEFAULT false,
  is_registered     boolean NOT NULL DEFAULT false,      -- the national address used in KYB
  deleted_at        timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_addresses_short CHECK (short_address IS NULL OR short_address ~ '^[A-Z]{4}[0-9]'),
  CONSTRAINT ck_addresses_postal CHECK (postal_code IS NULL OR country_code <> 'SA' OR postal_co
);

```

```

CREATE INDEX ix_addresses_org ON org.addresses (organization_id) WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX ux_addresses_default ON org.addresses (organization_id) WHERE is_default AND

```

```

CREATE TABLE org.business_profiles (
  organization_id   uuid PRIMARY KEY REFERENCES org.organizations(id),
  legal_name        text NOT NULL,
  trade_name        text,
  cr_number         text NOT NULL,

```

```

cr_expiry            date,
vat_number           text,
registered_address_id  uuid REFERENCES org.addresses(id),
contact_phone        text,
contact_email         citext,
verification_status   core.verification_status NOT NULL DEFAULT 'DRAFT',
verified_at          timestampz,
version              integer NOT NULL DEFAULT 1,
created_at            timestampz NOT NULL DEFAULT now(),
updated_at            timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_business_cr CHECK (cr_number ~ '^[0-9]{10}$'),
CONSTRAINT ck_business_vat CHECK (vat_number IS NULL OR vat_number ~ '^3[0-9]{13}3$')
);

-- A CR can be approved on one organization only; duplicates in review are flagged, not blocked
CREATE UNIQUE INDEX ux_business_cr_approved ON org.business_profiles (cr_number) WHERE verification_status = 'APPROVED';
CREATE INDEX ix_business_cr ON org.business_profiles (cr_number);

CREATE TABLE org.provider_activities (
  id                uuid PRIMARY KEY,
  organization_id    uuid NOT NULL REFERENCES org.organizations(id),
  activity           core.provider_activity_type NOT NULL,
  status            core.activity_status NOT NULL DEFAULT 'PENDING',
  approved_at        timestampz,
  suspended_at       timestampz,
  suspension_reason  text,                                -- e.g. LICENSE_EXPIRED (source SYSTEM)
  notes              text,
  created_at         timestampz NOT NULL DEFAULT now(),
  updated_at         timestampz NOT NULL DEFAULT now(),
  UNIQUE (organization_id, activity)
);

CREATE INDEX ix_provider_activities_matching ON org.provider_activities (activity) WHERE status = 'PENDING';

CREATE TABLE org.service_areas (
  id                uuid PRIMARY KEY,
  provider_activity_id  uuid NOT NULL REFERENCES org.provider_activities(id) ON DELETE CASCADE,
  area_type          core.service_area_type NOT NULL,
  city_id            uuid REFERENCES ref.cities(id),
  region_code        text REFERENCES ref.regions(code),
  country_code        char(2) REFERENCES ref.countries(code),
  port_id            uuid REFERENCES ref.ports(id),
  checkpoint_id       uuid REFERENCES ref.customs_checkpoints(id),
  origin_country_code char(2) REFERENCES ref.countries(code),    -- LANE
  destination_country_code char(2) REFERENCES ref.countries(code), -- LANE

```

```

created_at          timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_service_area_shape CHECK (
    (area_type = 'CITY'          AND city_id IS NOT NULL) OR
    (area_type = 'REGION'       AND region_code IS NOT NULL) OR
    (area_type = 'COUNTRY'      AND country_code IS NOT NULL) OR
    (area_type = 'PORT'         AND port_id IS NOT NULL) OR
    (area_type = 'CHECKPOINT'   AND checkpoint_id IS NOT NULL) OR
    (area_type = 'LANE'         AND origin_country_code IS NOT NULL AND destination_country_code :
);

CREATE INDEX ix_service_areas_activity ON org.service_areas (provider_activity_id);
CREATE INDEX ix_service_areas_city ON org.service_areas (city_id) WHERE city_id IS NOT NULL;
CREATE INDEX ix_service_areas_checkpoint ON org.service_areas (checkpoint_id) WHERE checkpoint_id IS NOT NULL;
CREATE INDEX ix_service_areas_lane ON org.service_areas (origin_country_code, destination_country_code);

CREATE TABLE org.licenses (
    id                uuid PRIMARY KEY,
    organization_id   uuid NOT NULL REFERENCES org.organizations(id),
    license_type      core.license_type NOT NULL,
    related_activity  core.provider_activity_type,
    number            text NOT NULL,
    issued_at         date,
    expires_at        date,
    document_id       uuid NOT NULL,                -- FK to files.documents (§21)
    status            core.license_status NOT NULL DEFAULT 'PENDING',
    last_reminder_at  timestampz,
    version           integer NOT NULL DEFAULT 1,
    created_at        timestampz NOT NULL DEFAULT now(),
    updated_at        timestampz NOT NULL DEFAULT now(),
    CONSTRAINT ck_licenses_dates CHECK (expires_at IS NULL OR issued_at IS NULL OR expires_at > issued_at);
);

CREATE INDEX ix_licenses_org ON org.licenses (organization_id);
CREATE INDEX ix_licenses_expiry ON org.licenses (expires_at) WHERE status = 'VALID' AND expires_at < now();

CREATE TABLE org.bank_accounts (
    id                uuid PRIMARY KEY,
    organization_id   uuid NOT NULL REFERENCES org.organizations(id),
    iban_enc          bytea NOT NULL,                -- AES-256-GCM, KMS envelope
    iban_last4        char(4) NOT NULL,
    iban_fingerprint  bytea NOT NULL,                -- HMAC(pepper, IBAN): duplicate detect:
    bank_name         text NOT NULL,
    account_holder_name text NOT NULL,
    proof_document_id uuid,                          -- FK to files.documents (§21)
    status            core.bank_account_status NOT NULL DEFAULT 'PENDING',

```

```

verified_at          timestampz,
is_default            boolean NOT NULL DEFAULT false,
payout_hold_until    timestampz,                -- 48 h hold after a change (anti-fraud)
deleted_at           timestampz,
created_at           timestampz NOT NULL DEFAULT now(),
updated_at           timestampz NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX ux_bank_accounts_default ON org.bank_accounts (organization_id) WHERE is_def
CREATE INDEX ix_bank_accounts_fingerprint ON org.bank_accounts (iban_fingerprint);

CREATE TABLE org.verification_cases (
  id                  uuid PRIMARY KEY,
  organization_id     uuid NOT NULL REFERENCES org.organizations(id),
  workspace           core.workspace_type NOT NULL,
  status             core.verification_status NOT NULL DEFAULT 'DRAFT',
  submitted_at        timestampz,
  decided_at         timestampz,
  assigned_staff_id   uuid REFERENCES identity.staff_users(id),
  reviewer_staff_id   uuid REFERENCES identity.staff_users(id),
  decision_reason     text,
  resubmission_count  integer NOT NULL DEFAULT 0,
  version            integer NOT NULL DEFAULT 1,
  created_at          timestampz NOT NULL DEFAULT now(),
  updated_at          timestampz NOT NULL DEFAULT now()
);

-- One open case per organization and workspace
CREATE UNIQUE INDEX ux_verification_open ON org.verification_cases (organization_id, workspace)
  WHERE status IN ('DRAFT', 'SUBMITTED', 'UNDER_REVIEW', 'CHANGES_REQUESTED');
CREATE INDEX ix_verification_queue ON org.verification_cases (status, submitted_at) WHERE statu:

CREATE TABLE org.verification_items (
  id                  uuid PRIMARY KEY,
  case_id            uuid NOT NULL REFERENCES org.verification_cases(id) ON DELETE CASCADE,
  item_type          core.verification_item_type NOT NULL,
  ref_id             uuid NOT NULL,                -- organization / license / bank account
  status            core.verification_item_status NOT NULL DEFAULT 'PENDING',
  reason_code        text,                        -- ref.reason_codes (domain KYB)
  reason_note        text,
  auto_check_result   jsonb,                      -- Wathq / National Address results (Phi
  decided_by_staff_id uuid REFERENCES identity.staff_users(id),
  decided_at         timestampz,
  created_at          timestampz NOT NULL DEFAULT now(),
  updated_at          timestampz NOT NULL DEFAULT now(),

```

```

    UNIQUE (case_id, item_type, ref_id)
);

CREATE TABLE org.terms_documents (
    id                uuid PRIMARY KEY,
    audience          core.terms_audience NOT NULL,
    version           text NOT NULL,          -- semver, e.g. 1.2.0
    locale            core.app_locale NOT NULL,
    title             text NOT NULL,
    body_markdown     text NOT NULL,
    summary_items     jsonb NOT NULL DEFAULT '[]', -- cards shown on A07
    status            core.terms_status NOT NULL DEFAULT 'DRAFT',
    requires_reconsent boolean NOT NULL DEFAULT false,
    legal_signoff_by  text,
    legal_signoff_at  timestampz,
    legal_signoff_file_id uuid,              -- FK to files.files (§21)
    is_current        boolean NOT NULL DEFAULT false,
    published_at      timestampz,
    published_by_staff_id uuid REFERENCES identity.staff_users(id),
    created_by_staff_id uuid REFERENCES identity.staff_users(id),
    created_at        timestampz NOT NULL DEFAULT now(),
    updated_at        timestampz NOT NULL DEFAULT now(),
    UNIQUE (audience, locale, version),
    CONSTRAINT ck_terms_published_signoff CHECK (status <> 'PUBLISHED' OR legal_signoff_at IS NOT
);

CREATE UNIQUE INDEX ux_terms_current ON org.terms_documents (audience, locale) WHERE is_current;

-- Append-only consent audit trail
CREATE TABLE org.consents (
    id                uuid PRIMARY KEY,
    user_id           uuid NOT NULL REFERENCES identity.users(id),
    organization_id    uuid NOT NULL REFERENCES org.organizations(id),
    workspace         core.workspace_type NOT NULL,
    consent_key       core.consent_key NOT NULL,
    terms_document_id uuid REFERENCES org.terms_documents(id),
    terms_version     text,
    context_type      text,                  -- SERVICE_REQUEST | QUOTE | ORDER | PRO
    context_id        uuid,
    context_reference  text,                -- LX-260148, PO-260083
    accepted_at       timestampz NOT NULL DEFAULT now(),
    ip               inet,
    device_id         text,
    user_agent        text

```

```

);
CREATE INDEX ix_consents_org_key ON org.consents (organization_id, consent_key, accepted_at DESC);
CREATE INDEX ix_consents_context ON org.consents (context_id) WHERE context_id IS NOT NULL;
CREATE INDEX ix_consents_terms ON org.consents (terms_document_id) WHERE terms_document_id IS NOT NULL;
-- Same consent is recorded once per user, key, context and terms version (idempotent POST /consents)
CREATE UNIQUE INDEX ux_consents_once ON org.consents (user_id, consent_key,
  coalesce(context_id, '00000000-0000-0000-0000-000000000000'::uuid),
  coalesce(terms_document_id, '00000000-0000-0000-0000-000000000000'::uuid));

```

9. files : files and documents

-- Storage layer: one row per uploaded or generated object

```
CREATE TABLE files.files (
  id                uuid PRIMARY KEY,
  bucket            text NOT NULL,                -- logix-private-{env} | logix-public-{env}
  storage_key       text NOT NULL UNIQUE,
  purpose           core.file_purpose NOT NULL,
  mime_type         text NOT NULL,
  size_bytes        bigint NOT NULL CHECK (size_bytes > 0),
  sha256            bytea,
  original_name     text,
  organization_id   uuid REFERENCES org.organizations(id),
  uploaded_by_user_id uuid REFERENCES identity.users(id),
  uploaded_by_staff_id uuid REFERENCES identity.staff_users(id),
  upload_completed_at timestampz,
  scan_status       core.scan_status NOT NULL DEFAULT 'PENDING',
  scanned_at        timestampz,
  width             integer,
  height            integer,
  page_count        integer,
  thumbnails        jsonb,                        -- {"320": "key", "1024": "key"}
  captured_at       timestampz,                    -- evidence photos (sent by the app)
  captured_location geography(Point, 4326),
  deleted_at        timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_files_uploader CHECK (
    uploaded_by_user_id IS NOT NULL OR uploaded_by_staff_id IS NOT NULL OR purpose IN ('EXPORT', 'EXPORT_PREVIEW')
  );

CREATE INDEX ix_files_pending_scan ON files.files (created_at) WHERE scan_status = 'PENDING';
CREATE INDEX ix_files_org ON files.files (organization_id, created_at DESC);
```

-- Business layer: typed, versioned, reviewable document attached to an owner

```
CREATE TABLE files.documents (
  id                uuid PRIMARY KEY,
  owner_type        core.document_owner_type NOT NULL,
  owner_id          uuid NOT NULL,                -- polymorphic; ownership checked by trigger
  document_type_code text NOT NULL REFERENCES ref.document_types(code),
  custom_name       text,                        -- required for OTHER ("Add and name a document")
  status            core.document_status NOT NULL DEFAULT 'MISSING',
  requirement        core.requirement_level,      -- from ref.document_requirements when document_type_code = 'OTHER'
```

```

requested_by_org_id    uuid REFERENCES org.organizations(id), -- broker-requested documents
request_note           text,
reviewer_scope         core.document_reviewer_scope NOT NULL,
current_version_id     uuid,                                -- FK to files.document_versions (defer
rejection_reason_code  text,
rejection_note         text,
reviewed_by_user_id    uuid REFERENCES identity.users(id),
reviewed_by_staff_id   uuid REFERENCES identity.staff_users(id),
reviewed_at            timestampz,
expiry_date            date,
visibility              core.document_visibility NOT NULL DEFAULT 'ALL_PARTIES',
version                integer NOT NULL DEFAULT 1,
created_at             timestampz NOT NULL DEFAULT now(),
updated_at             timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_documents_other_named CHECK (document_type_code <> 'OTHER' OR custom_name IS NOT NULL),
CONSTRAINT ck_documents_has_version CHECK (status = 'MISSING' OR current_version_id IS NOT NULL),
);

CREATE INDEX ix_documents_owner ON files.documents (owner_type, owner_id);
CREATE INDEX ix_documents_review_queue ON files.documents (reviewer_scope, status) WHERE status = 'PENDING';
CREATE INDEX ix_documents_expiry ON files.documents (expiry_date) WHERE expiry_date IS NOT NULL;
-- One document per type per owner (except user-named "OTHER" documents)
CREATE UNIQUE INDEX ux_documents_owner_type ON files.documents (owner_type, owner_id, document_type_code)
WHERE document_type_code <> 'OTHER';

-- Immutable versions (replacing a document adds a version)
CREATE TABLE files.document_versions (
  id                uuid PRIMARY KEY,
  document_id       uuid NOT NULL REFERENCES files.documents(id),
  file_id           uuid NOT NULL REFERENCES files.files(id),
  version_no        integer NOT NULL CHECK (version_no >= 1),
  uploaded_by_user_id  uuid REFERENCES identity.users(id),
  uploaded_by_org_id  uuid REFERENCES org.organizations(id),
  uploaded_by_staff_id uuid REFERENCES identity.staff_users(id),
  note              text,
  created_at        timestampz NOT NULL DEFAULT now(),
  UNIQUE (document_id, version_no)
);

-- Review decision history (append-only): powers "Versions + decisions" and rejection reasons in the UI
CREATE TABLE files.document_reviews (
  id                uuid PRIMARY KEY,
  document_id       uuid NOT NULL REFERENCES files.documents(id),
  document_version_id  uuid NOT NULL REFERENCES files.document_versions(id),

```

```

decision                core.review_decision NOT NULL,
reason_code              text,
note                    text,
reviewer_user_id         uuid REFERENCES identity.users(id),
reviewer_org_id          uuid REFERENCES org.organizations(id),
reviewer_staff_id        uuid REFERENCES identity.staff_users(id),
created_at               timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_document_reviews_reviewer CHECK (reviewer_user_id IS NOT NULL OR reviewer_staff_
);
CREATE INDEX ix_document_reviews_doc ON files.document_reviews (document_id, created_at DESC);

```

10. svc : requests, quotes, orders

The shared engine for Shipping, Transport, Warehousing and Customs. The request and its order share one LX- reference.

10.1 Requests

```
CREATE TABLE svc.service_requests (
  id                uuid PRIMARY KEY,
  reference         text NOT NULL UNIQUE,
  service_type      core.service_type NOT NULL,
  customer_org_id   uuid NOT NULL REFERENCES org.organizations(id),
  created_by_user_id uuid NOT NULL REFERENCES identity.users(id),
  status            core.request_status NOT NULL DEFAULT 'DRAFT',
  current_step      text, -- last saved wizard step (mode, route,
  origin_city_id    uuid REFERENCES ref.cities(id),
  destination_city_id uuid REFERENCES ref.cities(id),
  origin_country_code char(2) REFERENCES ref.countries(code),
  destination_country_code char(2) REFERENCES ref.countries(code),
  origin_summary    text, -- "Shanghai, China"
  destination_summary text, -- "Jeddah Islamic Port"
  service_date      date, -- ready / transport / storage start date
  notes             text,
  other_requirements text,
  accuracy_consent_id uuid REFERENCES org.consent(id),
  parent_order_id   uuid, -- linked request (FK §21)
  link_type         core.order_link_type,
  duplicated_from_request_id uuid REFERENCES svc.service_requests(id), -- "Reuse request details"
  submitted_at      timestampz,
  expires_at        timestampz,
  cancelled_at      timestampz,
  cancel_reason_code text,
  matched_provider_count integer NOT NULL DEFAULT 0,
  quote_count       integer NOT NULL DEFAULT 0,
  first_quote_at    timestampz,
  flags             text[] NOT NULL DEFAULT '{}', -- ZERO_MATCH, ZERO_QUOTE
  version           integer NOT NULL DEFAULT 1,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_requests_reference CHECK (reference ~ '^[A-Z]{2,4}-[0-9]{6,}$'),
  CONSTRAINT ck_requests_submitted CHECK (status IN ('DRAFT', 'CANCELLED') OR submitted_at IS NULL),
  CONSTRAINT ck_requests_consent CHECK (status IN ('DRAFT', 'CANCELLED') OR accuracy_consent_id IS NULL),
  CONSTRAINT ck_requests_link CHECK ((parent_order_id IS NULL) = (link_type IS NULL))
);

CREATE INDEX ix_requests_customer ON svc.service_requests (customer_org_id, created_at DESC);
CREATE INDEX ix_requests_status ON svc.service_requests (status, service_type);
CREATE INDEX ix_requests_expiry ON svc.service_requests (expires_at) WHERE status IN ('SUBMITTED', 'CANCELLED');
CREATE INDEX ix_requests_origin_city ON svc.service_requests (origin_city_id) WHERE status IN ('SUBMITTED', 'CANCELLED');
```

```

CREATE INDEX ix_requests_flags ON svc.service_requests USING gin (flags);
CREATE INDEX ix_requests_drafts ON svc.service_requests (updated_at) WHERE status = 'DRAFT'; --

CREATE TABLE svc.shipping_request_details (
  request_id          uuid PRIMARY KEY REFERENCES svc.service_requests(id) ON DELETE CASCADE,
  mode                core.shipping_mode,
  is_express           boolean NOT NULL DEFAULT false,
  is_door_to_door      boolean NOT NULL DEFAULT false,
  trade_direction      core.trade_direction,
  origin_country_code  char(2) REFERENCES ref.countries(code),
  origin_port_id       uuid REFERENCES ref.ports(id),
  origin_city_id       uuid REFERENCES ref.cities(id),
  destination_country_code char(2) REFERENCES ref.countries(code),
  destination_port_id  uuid REFERENCES ref.ports(id),
  destination_city_id  uuid REFERENCES ref.cities(id),
  cargo_ready_date     date,
  commodity_code       text REFERENCES ref.commodity_categories(code),
  commodity_other      text,
  hs_code              text CHECK (hs_code IS NULL OR hs_code ~ '^[0-9]{6,12}$'),
  goods_description    text,
  container_type_code  text REFERENCES ref.container_types(code),
  container_size        core.container_size,
  container_count       smallint CHECK (container_count IS NULL OR container_count >= 1),
  temperature_c        numeric(4,1),
  weight_kg            numeric(12,2) CHECK (weight_kg IS NULL OR weight_kg > 0),
  volume_cbm           numeric(10,3) CHECK (volume_cbm IS NULL OR volume_cbm > 0),
  package_count        integer CHECK (package_count IS NULL OR package_count >= 1),
  packages             jsonb, -- [{"lengthCm":...,"widthCm":...,"heightCm":...}]
  pallet_count         integer,
  pickup_address_id     uuid REFERENCES org.addresses(id),
  delivery_address_id   uuid REFERENCES org.addresses(id),
  pickup_window_start   time,
  pickup_window_end     time,
  sender_contact        jsonb, -- {"name":...,"phone":...}
  recipient_contact     jsonb,
  access_notes          text,
  handling_requirements text,
  border_instructions   text,
  wants_customs_clearance boolean NOT NULL DEFAULT false,
  wants_insurance       boolean NOT NULL DEFAULT false,
  extras               jsonb NOT NULL DEFAULT '{}', -- urgent priority, temporary storage.
  CONSTRAINT ck_shipping_window CHECK (pickup_window_start IS NULL OR pickup_window_end IS NULL OR pickup_window_start <= pickup_window_end),
  CONSTRAINT ck_shipping_express_air CHECK (NOT is_express OR mode IS NULL OR mode <> 'SEA')
);

```

```
);
```

```
CREATE TABLE svc.transport_request_details (  
  request_id          uuid PRIMARY KEY REFERENCES svc.service_requests(id) ON DELETE CASCADE  
  scope              core.transport_scope,  
  vehicle_type_code   text REFERENCES ref.vehicle_types(code),  
  vehicle_description text, -- required when vehicle type is OTHER  
  pickup_address_id   uuid REFERENCES org.addresses(id),  
  pickup_location     geography(Point, 4326),  
  pickup_label        text, -- "Riyadh Industrial City"  
  pickup_contact      jsonb,  
  dropoff_address_id  uuid REFERENCES org.addresses(id),  
  dropoff_location    geography(Point, 4326),  
  dropoff_label       text,  
  dropoff_contact     jsonb,  
  destination_country_code char(2) REFERENCES ref.countries(code), -- cross-border  
  transport_date      date,  
  time_window_start   time,  
  time_window_end     time,  
  commodity_code      text REFERENCES ref.commodity_categories(code),  
  commodity_other     text,  
  weight_value        numeric(12,2) CHECK (weight_value IS NULL OR weight_value > 0),  
  weight_unit         core.weight_unit,  
  quantity            numeric(12,2) CHECK (quantity IS NULL OR quantity > 0),  
  quantity_unit_code  text REFERENCES ref.units_of_measure(code),  
  loading_assistance  core.loading_assistance,  
  temperature_c       numeric(4,1), -- reefer only  
  special_handling    text,  
  border_instructions text,  
  vehicles_count      smallint CHECK (vehicles_count IS NULL OR vehicles_count >= 1), --  
  extras              jsonb NOT NULL DEFAULT '{}', -- cross-docking, temporary storage, l  
  route_distance_km   numeric(8,1),  
  route_duration_min  integer,  
  route_polyline      text  
);
```

```
CREATE TABLE svc.warehousing_request_details (  
  request_id          uuid PRIMARY KEY REFERENCES svc.service_requests(id) ON DELETE CASCADE  
  city_id            uuid REFERENCES ref.cities(id),  
  storage_type_code   text REFERENCES ref.storage_types(code),  
  start_date         date,  
  end_date           date,  
  space_value        numeric(10,2) CHECK (space_value IS NULL OR space_value > 0),
```

```

space_unit            core.space_unit,
commodity_code        text REFERENCES ref.commodity_categories(code),
commodity_other       text,
quantity             numeric(12,2),
quantity_unit_code    text REFERENCES ref.units_of_measure(code),
weight_kg            numeric(12,2),
temperature_min_c     numeric(4,1),
temperature_max_c     numeric(4,1),
handling_services     text[] NOT NULL DEFAULT '{}', -- ref.handling_services codes
transport_to_warehouse core.warehouse_transport,
special_requirements  text,
CONSTRAINT ck_warehousing_dates CHECK (start_date IS NULL OR end_date IS NULL OR end_date > start_date),
CONSTRAINT ck_warehousing_temp CHECK (temperature_min_c IS NULL OR temperature_max_c IS NULL OR temperature_min_c < temperature_max_c);
);

```

```

CREATE TABLE svc.customs_request_details (
request_id            uuid PRIMARY KEY REFERENCES svc.service_requests(id) ON DELETE CASCADE,
movement             core.customs_movement,
checkpoint_type       core.checkpoint_type,
checkpoint_id         uuid REFERENCES ref.customs_checkpoints(id),
cargo_type            text, -- "Dry container"
cargo_quantity        integer,
bill_of_lading_no     text,
hs_code              text CHECK (hs_code IS NULL OR hs_code ~ '^[0-9]{6,12}$'),
goods_description     text,
arrival_port_id       uuid REFERENCES ref.ports(id), -- import / transit
departure_port_id     uuid REFERENCES ref.ports(id), -- export
shipment_quantity     text,
transit_destination   text, -- transit only
exit_checkpoint_id    uuid REFERENCES ref.customs_checkpoints(id), -- transit only
notes                text
);

```

10.2 Matching, clarifications and quotes

```
CREATE TABLE svc.request_matches (  
  id                uuid PRIMARY KEY,  
  request_id        uuid NOT NULL REFERENCES svc.service_requests(id),  
  provider_org_id   uuid NOT NULL REFERENCES org.organizations(id),  
  activity           core.provider_activity_type NOT NULL,  
  notified_at       timestampz,  
  viewed_at         timestampz,  
  quoted_at         timestampz,  
  declined_at       timestampz,  
  decline_reason_code text,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  UNIQUE (request_id, provider_org_id)  
);  
  
CREATE INDEX ix_matches_provider_feed ON svc.request_matches (provider_org_id, created_at DESC);  
  
-- Public pre-award Q&A (answers visible to all matched providers, anonymized)  
CREATE TABLE svc.request_questions (  
  id                uuid PRIMARY KEY,  
  request_id        uuid NOT NULL REFERENCES svc.service_requests(id),  
  provider_org_id   uuid NOT NULL REFERENCES org.organizations(id),  
  asked_by_user_id  uuid NOT NULL REFERENCES identity.users(id),  
  question          text NOT NULL,  
  question_masked   boolean NOT NULL DEFAULT false, -- contact details masked (D-16)  
  answer            text,  
  answer_masked     boolean NOT NULL DEFAULT false,  
  answered_by_user_id uuid REFERENCES identity.users(id),  
  answered_at       timestampz,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  updated_at        timestampz NOT NULL DEFAULT now()  
);  
  
CREATE INDEX ix_questions_request ON svc.request_questions (request_id, created_at);  
  
CREATE TABLE svc.quotes (  
  id                uuid PRIMARY KEY,  
  request_id        uuid NOT NULL REFERENCES svc.service_requests(id),  
  provider_org_id   uuid NOT NULL REFERENCES org.organizations(id),  
  status            core.quote_status NOT NULL DEFAULT 'SUBMITTED',  
  revision          integer NOT NULL DEFAULT 1,  
  currency          core.currency_code NOT NULL,  
  service_fee       core.halalas NOT NULL,  
  charges_total     core.halalas NOT NULL DEFAULT 0,
```

```

pass_through_total      core.halalas NOT NULL DEFAULT 0,    -- non-taxable, non-commission:
subtotal                core.halalas NOT NULL,
vat_rate_bps            core.bps NOT NULL,
vat_amount              core.halalas NOT NULL,
total_amount            core.halalas NOT NULL,          -- what the customer pays
commission_category      core.commission_category NOT NULL,
commission_rate_bps_preview core.bps NOT NULL,
commission_amount_preview core.halalas NOT NULL,
commission_vat_preview   core.halalas NOT NULL DEFAULT 0,
net_to_provider_preview  core.halalas NOT NULL,
eta_date                date,
duration_days            numeric(5,1),
valid_until              timestampz NOT NULL,
scope_included           text,
exclusions               text,
notes                   text,
notes_masked             boolean NOT NULL DEFAULT false,
internal_cost            core.halalas,                  -- PROVIDER-PRIVATE: never serialized
target_margin            core.signed_halalas,            -- PROVIDER-PRIVATE
submitted_by_user_id     uuid NOT NULL REFERENCES identity.users(id),
submitted_at             timestampz NOT NULL DEFAULT now(),
withdrawn_at             timestampz,
accepted_at              timestampz,
version                  integer NOT NULL DEFAULT 1,
created_at               timestampz NOT NULL DEFAULT now(),
updated_at               timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_quotes_subtotal CHECK (subtotal = service_fee + charges_total + pass_through_tot
CONSTRAINT ck_quotes_total   CHECK (total_amount = subtotal + vat_amount),
CONSTRAINT ck_quotes_validity CHECK (valid_until > submitted_at)
);

-- One live quote per provider per request (revisions update the same row and bump `revision`)
CREATE UNIQUE INDEX ux_quotes_live ON svc.quotes (request_id, provider_org_id)
  WHERE status IN ('SUBMITTED', 'ACCEPTED_PENDING_PAYMENT', 'ACCEPTED');
CREATE INDEX ix_quotes_request ON svc.quotes (request_id, status);
CREATE INDEX ix_quotes_provider ON svc.quotes (provider_org_id, status, submitted_at DESC);
CREATE INDEX ix_quotes_expiry ON svc.quotes (valid_until) WHERE status = 'SUBMITTED';

CREATE TABLE svc.quote_charges (
  id          uuid PRIMARY KEY,
  quote_id    uuid NOT NULL REFERENCES svc.quotes(id) ON DELETE CASCADE,
  label       text NOT NULL,          -- "Loading / unloading"
  amount      core.halalas NOT NULL,
  is_taxable   boolean NOT NULL DEFAULT true,

```

```
is_pass_through  boolean NOT NULL DEFAULT false,  
sort_order       smallint NOT NULL DEFAULT 0,  
CONSTRAINT ck_quote_charges_passthrough CHECK (NOT (is_pass_through AND is_taxable))  
);
```

-- Snapshot of every revision shown to the customer (audit of price changes)

```
CREATE TABLE svc.quote_revisions (  
  id          uuid PRIMARY KEY,  
  quote_id    uuid NOT NULL REFERENCES svc.quotes(id),  
  revision    integer NOT NULL,  
  snapshot    jsonb NOT NULL,                -- customer-visible fields only  
  created_at  timestampz NOT NULL DEFAULT now(),  
  UNIQUE (quote_id, revision)  
);
```

10.3 Orders and shared execution

```
CREATE TABLE svc.orders (  
  id                uuid PRIMARY KEY,  
  reference         text NOT NULL UNIQUE,          -- same value as the request reference  
  service_type     core.service_type NOT NULL,  
  request_id       uuid NOT NULL UNIQUE REFERENCES svc.service_requests(id),  
  accepted_quote_id uuid NOT NULL UNIQUE REFERENCES svc.quotes(id),  
  customer_org_id  uuid NOT NULL REFERENCES org.organizations(id),  
  provider_org_id  uuid NOT NULL REFERENCES org.organizations(id),  
  status           core.order_status NOT NULL DEFAULT 'PENDING_PAYMENT',  
  resume_status    core.order_status,              -- restored if a cancellation request is  
  payment_status   core.payment_summary_status NOT NULL DEFAULT 'UNPAID',  
  settlement_status core.settlement_status,  
  -- amounts: snapshot of the accepted quote  
  currency         core.currency_code NOT NULL,  
  service_fee      core.halalas NOT NULL,  
  charges_total    core.halalas NOT NULL DEFAULT 0,  
  pass_through_total core.halalas NOT NULL DEFAULT 0,  
  subtotal         core.halalas NOT NULL,  
  vat_rate_bps     core.bps NOT NULL,  
  vat_amount       core.halalas NOT NULL,  
  total_amount     core.halalas NOT NULL,  
  additional_paid_total core.halalas NOT NULL DEFAULT 0, -- approved and paid additional charges  
  -- commission snapshot (rule version frozen at order creation)  
  commission_category core.commission_category NOT NULL,  
  commission_rule_id  uuid NOT NULL,              -- FK fin.commission_rules (§21)  
  commission_rate_bps core.bps NOT NULL,  
  commission_amount   core.halalas NOT NULL,  
  commission_vat_amount core.halalas NOT NULL DEFAULT 0,  
  net_to_provider     core.halalas NOT NULL,  
  -- execution state (denormalized for lists and queues)  
  current_milestone_code text,  
  next_action_code     text,                      -- REVIEW_DOCUMENTS, CONFIRM_RECEIPT, B  
  next_action_actor    core.workspace_type,  
  next_action_due_at   timestampz,  
  eta_at              timestampz,  
  last_update_at      timestampz,  
  confirmed_at        timestampz,                -- payment verified by the gateway  
  scheduled_at        timestampz,                -- provider confirmed readiness  
  started_at          timestampz,  
  pod_at              timestampz,  
  accepted_at         timestampz,                -- RC / clearance confirmation / reconcili
```

```

auto_accept_at          timestamptz,          -- D-10 timer
auto_accept_paused      boolean NOT NULL DEFAULT false,
completed_at           timestamptz,
cancelled_at           timestamptz,
version                integer NOT NULL DEFAULT 1,
created_at             timestamptz NOT NULL DEFAULT now(),
updated_at             timestamptz NOT NULL DEFAULT now(),
CONSTRAINT ck_orders_reference CHECK (reference ~ '^[A-Z]{2,4}-[0-9]{6,}$'),
CONSTRAINT ck_orders_subtotal CHECK (subtotal = service_fee + charges_total + pass_through_t
CONSTRAINT ck_orders_total CHECK (total_amount = subtotal + vat_amount),
CONSTRAINT ck_orders_parties CHECK (customer_org_id <> provider_org_id),
CONSTRAINT ck_orders_wh_states CHECK (status NOT IN ('ACTIVE', 'CLOSING') OR service_type = 'I
CONSTRAINT ck_orders_completed CHECK (status <> 'COMPLETED' OR completed_at IS NOT NULL)
);

CREATE INDEX ix_orders_customer ON svc.orders (customer_org_id, created_at DESC);
CREATE INDEX ix_orders_provider ON svc.orders (provider_org_id, status, created_at DESC);
CREATE INDEX ix_orders_status ON svc.orders (status, service_type);
CREATE INDEX ix_orders_auto_accept ON svc.orders (auto_accept_at)
  WHERE status = 'AWAITING_ACCEPTANCE' AND NOT auto_accept_paused;
CREATE INDEX ix_orders_readiness_overdue ON svc.orders (confirmed_at) WHERE status = 'CONFIRMED
CREATE INDEX ix_orders_stale ON svc.orders (last_update_at) WHERE status IN ('SCHEDULED', 'IN_PI

-- Links between orders / requests / purchase orders / insurance (e.g. customs for a shipment, c
-- Created when the linked request is submitted; child_order_id is filled when that request com
CREATE TABLE svc.order_links (
  id                uuid PRIMARY KEY,
  link_type         core.order_link_type NOT NULL,
  parent_order_id   uuid REFERENCES svc.orders(id),
  parent_purchase_order_id  uuid,          -- FK mkt.purchase_orders (§21)
  child_request_id  uuid REFERENCES svc.service_requests(id),
  child_order_id    uuid REFERENCES svc.orders(id),
  child_purchase_order_id  uuid,          -- FK mkt.purchase_orders (§21): replace
  child_insurance_request_id  uuid,      -- FK ops.insurance_requests (§21)
  created_by_user_id  uuid REFERENCES identity.users(id),
  created_at         timestamptz NOT NULL DEFAULT now(),
  updated_at        timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT ck_order_links_parent CHECK (num_nonnulls(parent_order_id, parent_purchase_order_id)
  CONSTRAINT ck_order_links_child CHECK (num_nonnulls(child_request_id, child_order_id, child_purchase_order_id,
  child_insurance_request_id) >= 1)
);

CREATE INDEX ix_order_links_parent_order ON svc.order_links (parent_order_id) WHERE parent_order_id IS NOT NULL;
CREATE INDEX ix_order_links_parent_po ON svc.order_links (parent_purchase_order_id) WHERE parent_purchase_order_id IS NOT NULL;
CREATE INDEX ix_order_links_child_order ON svc.order_links (child_order_id) WHERE child_order_id IS NOT NULL;

```

```
CREATE INDEX ix_order_links_child_request ON svc.order_links (child_request_id) WHERE child_request_id IS NOT NULL;

-- Current milestone state, created from the service template when the order is confirmed
CREATE TABLE svc.order_milestones (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL REFERENCES svc.orders(id),
  code              text NOT NULL,                -- BOOKING_CONFIRMED, CARGO_COLLECTED, ...
  sequence          smallint NOT NULL,
  is_optional       boolean NOT NULL DEFAULT false,
  status            core.milestone_status NOT NULL DEFAULT 'PENDING',
  estimated_at      timestampz,
  occurred_at       timestampz,
  source            core.update_source,
  recorded_by_user_id  uuid REFERENCES identity.users(id),
  recorded_by_staff_id  uuid REFERENCES identity.staff_users(id),
  note              text,
  location          geography(Point, 4326),
  evidence_file_ids  uuid[] NOT NULL DEFAULT '{}',
  skipped_reason     text,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  UNIQUE (order_id, code),
  UNIQUE (order_id, sequence),
  CONSTRAINT ck_milestones_done    CHECK (status <> 'DONE' OR (occurred_at IS NOT NULL AND source IS NOT NULL)),
  CONSTRAINT ck_milestones_skipped CHECK (status <> 'SKIPPED' OR skipped_reason IS NOT NULL)
);
```

```

);
CREATE INDEX ix_order_events_order ON svc.order_events (order_id, occurred_at);

-- "No unapproved charges": extras need customer approval and their own payment (fin.payment_int
CREATE TABLE svc.additional_charges (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL REFERENCES svc.orders(id),
  proposed_by_org_id  uuid NOT NULL REFERENCES org.organizations(id),
  proposed_by_user_id  uuid REFERENCES identity.users(id),
  description        text NOT NULL,
  amount            core.halalas NOT NULL CHECK (amount > 0),
  is_taxable         boolean NOT NULL DEFAULT true,
  is_pass_through     boolean NOT NULL DEFAULT false, -- e.g. duties paid by the broker on the
  vat_amount         core.halalas NOT NULL DEFAULT 0,
  total_amount       core.halalas NOT NULL,
  status            core.additional_charge_status NOT NULL DEFAULT 'PROPOSED',
  decided_by_user_id  uuid REFERENCES identity.users(id),
  decided_at         timestampz,
  rejection_reason    text,
  paid_at            timestampz,
  version            integer NOT NULL DEFAULT 1,
  created_at         timestampz NOT NULL DEFAULT now(),
  updated_at         timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_charges_total      CHECK (total_amount = amount + vat_amount),
  CONSTRAINT ck_charges_passthrough CHECK (NOT (is_pass_through AND is_taxable))
);
CREATE INDEX ix_additional_charges_order ON svc.additional_charges (order_id, status);

-- Provider proof of delivery (POD). Never replaces customer acceptance.
CREATE TABLE svc.completion_evidence (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL UNIQUE REFERENCES svc.orders(id),
  submitted_by_user_id  uuid NOT NULL REFERENCES identity.users(id),
  submitted_by_org_id  uuid NOT NULL REFERENCES org.organizations(id),
  source            core.update_source NOT NULL,
  recipient_name      text,
  handed_over_at      timestampz NOT NULL,
  photo_file_ids      uuid[] NOT NULL DEFAULT '{}',
  signature_file_id    uuid REFERENCES files.files(id),
  release_notice_document_id  uuid REFERENCES files.documents(id), -- customs POD
  note                text,
  location            geography(Point, 4326),
  created_at         timestampz NOT NULL DEFAULT now(),

```

```

    CONSTRAINT ck_pod_source CHECK (source IN ('PROVIDER', 'DRIVER', 'BROKER', 'ADMIN'))
);

-- Customer receipt confirmation (RC) for service orders, purchase orders and warehouse release:
CREATE TABLE svc.receipt_confirmations (
    id                uuid PRIMARY KEY,
    order_id          uuid REFERENCES svc.orders(id),
    purchase_order_id uuid,                                -- FK mkt.purchase_orders (§21)
    release_request_id uuid,                                -- FK warehouse.release_requests (§21)
    condition         core.receipt_condition NOT NULL,
    unit_count_confirmed numeric(12,2),
    recipient_name    text,
    signature_file_id uuid REFERENCES files.files(id),
    photo_file_ids    uuid[] NOT NULL DEFAULT '{}',
    notes             text,
    is_auto           boolean NOT NULL DEFAULT false, -- auto-acceptance after POD (D-10), rec
    confirmed_by_user_id uuid REFERENCES identity.users(id),
    consent_id        uuid REFERENCES org.consent(id),
    confirmed_at      timestampz NOT NULL DEFAULT now(),
    created_at        timestampz NOT NULL DEFAULT now(),
    CONSTRAINT ck_rc_subject CHECK (num_nonnulls(order_id, purchase_order_id, release_request_id) > 0),
    CONSTRAINT ck_rc_actor CHECK (is_auto OR (confirmed_by_user_id IS NOT NULL AND consent_id IS NOT NULL)),
    CONSTRAINT ck_rc_issue_photos CHECK (condition = 'INTACT' OR cardinality(photo_file_ids) > 0)
);

CREATE UNIQUE INDEX ux_rc_order ON svc.receipt_confirmations (order_id) WHERE order_id IS NOT NULL;
CREATE UNIQUE INDEX ux_rc_po ON svc.receipt_confirmations (purchase_order_id) WHERE purchase_order_id IS NOT NULL;
CREATE UNIQUE INDEX ux_rc_release ON svc.receipt_confirmations (release_request_id) WHERE release_request_id IS NOT NULL;

-- Ratings for providers (service orders) and suppliers (purchase orders): one per completed order
CREATE TABLE svc.ratings (
    id                uuid PRIMARY KEY,
    order_id          uuid REFERENCES svc.orders(id),
    purchase_order_id uuid,                                -- FK mkt.purchase_orders (§21)
    rater_org_id      uuid NOT NULL REFERENCES org.organizations(id),
    rater_user_id     uuid NOT NULL REFERENCES identity.users(id),
    ratee_org_id      uuid NOT NULL REFERENCES org.organizations(id),
    stars             smallint NOT NULL CHECK (stars BETWEEN 1 AND 5),
    comment           text,
    status            core.rating_status NOT NULL DEFAULT 'PUBLISHED',
    moderated_by_staff_id uuid REFERENCES identity.staff_users(id),
    moderated_at      timestampz,
    created_at        timestampz NOT NULL DEFAULT now(),
    updated_at        timestampz NOT NULL DEFAULT now(),

```

```
CONSTRAINT ck_ratings_subject CHECK (num_nonnulls(order_id, purchase_order_id) = 1),
CONSTRAINT ck_ratings_self CHECK (rater_org_id <> ratee_org_id)
);
CREATE UNIQUE INDEX ux_ratings_order ON svc.ratings (order_id) WHERE order_id IS NOT NULL;
CREATE UNIQUE INDEX ux_ratings_po ON svc.ratings (purchase_order_id) WHERE purchase_order_id IS
CREATE INDEX ix_ratings_ratee ON svc.ratings (ratee_org_id) WHERE status = 'PUBLISHED';
```

11. freight

-- PS1 Book freight capacity

```
CREATE TABLE freight.bookings (  
  id                                uuid PRIMARY KEY,  
  order_id                        uuid NOT NULL UNIQUE REFERENCES svc.orders(id),  
  carrier_type                    core.carrier_type NOT NULL,  
  carrier_name                    text NOT NULL,          -- line, airline or road carrier  
  vessel_or_flight                text,  
  voyage                          text,  
  capacity_confirmed              boolean NOT NULL DEFAULT false,  
  etd                            timestamptz,  
  eta                            timestamptz,  
  booking_reference               text,  
  booking_confirmation_document_id uuid REFERENCES files.documents(id),  
  handling_partners               text,  
  notes                           text,  
  created_by_user_id              uuid NOT NULL REFERENCES identity.users(id),  
  version                         integer NOT NULL DEFAULT 1,  
  created_at                      timestamptz NOT NULL DEFAULT now(),  
  updated_at                      timestamptz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_bookings_dates CHECK (etd IS NULL OR eta IS NULL OR eta > etd)  
);
```

-- PS2 Execution references by mode (sea: B/L, containers, seals; air: AWB; land: vehicle)

```
CREATE TABLE freight.shipment_references (  
  order_id                        uuid PRIMARY KEY REFERENCES svc.orders(id),  
  bill_of_lading_number          text,  
  container_numbers              text[] NOT NULL DEFAULT '{}', -- ISO 6346 (check digit validated by tl  
  seal_numbers                   text[] NOT NULL DEFAULT '{}',  
  awb_number                     text CHECK (awb_number IS NULL OR awb_number ~ '^[0-9]{3}-?[0-9]{8}$').  
  parcel_tracking_ref            text,  
  vehicle_plate                  text,  
  driver_name                    text,  
  updated_by_user_id              uuid REFERENCES identity.users(id),  
  created_at                     timestamptz NOT NULL DEFAULT now(),  
  updated_at                     timestamptz NOT NULL DEFAULT now()  
);
```

```
CREATE INDEX ix_shipment_refs_bl ON freight.shipment_references (bill_of_lading_number) WHERE b:  
CREATE INDEX ix_shipment_refs_awb ON freight.shipment_references (awb_number) WHERE awb_number :  
CREATE INDEX ix_shipment_refs_containers ON freight.shipment_references USING gin (container_nur
```

12. transport

```
CREATE TABLE transport.vehicles (  
  id                                uuid PRIMARY KEY,  
  organization_id                  uuid NOT NULL REFERENCES org.organizations(id),  
  plate_number                     text NOT NULL,  
  vehicle_type_code               text NOT NULL REFERENCES ref.vehicle_types(code),  
  capacity_ton                    numeric(6,2),  
  is_reefer                       boolean NOT NULL DEFAULT false,  
  make_model                      text,  
  model_year                      smallint,  
  status                          core.vehicle_status NOT NULL DEFAULT 'ACTIVE',  
  operating_card_document_id       uuid REFERENCES files.documents(id),  
  operating_card_expiry            date,  
  insurance_document_id           uuid REFERENCES files.documents(id),  
  insurance_expiry                date,  
  deleted_at                     timestamptz,  
  created_at                     timestamptz NOT NULL DEFAULT now(),  
  updated_at                     timestamptz NOT NULL DEFAULT now()  
);  
  
CREATE UNIQUE INDEX ux_vehicles_plate ON transport.vehicles (organization_id, plate_number) WHERE :  
CREATE INDEX ix_vehicles_type ON transport.vehicles (organization_id, vehicle_type_code) WHERE :  
CREATE INDEX ix_vehicles_expiry ON transport.vehicles (least(operating_card_expiry, insurance_e  
  
-- A driver is a membership (role DRIVER) with licence data  
CREATE TABLE transport.drivers (  
  id                                uuid PRIMARY KEY,  
  membership_id                   uuid NOT NULL UNIQUE REFERENCES org.memberships(id),  
  organization_id                 uuid NOT NULL REFERENCES org.organizations(id),  
  user_id                        uuid NOT NULL REFERENCES identity.users(id),  
  license_number_enc              bytea,                                -- encrypted  
  license_number_last4            text,  
  license_expiry                 date,  
  license_document_id            uuid REFERENCES files.documents(id),  
  national_id_enc                 bytea,                                -- encrypted (national ID / Iqama)  
  national_id_last4              text,  
  status                         core.driver_status NOT NULL DEFAULT 'INVITED',  
  is_guest                       boolean NOT NULL DEFAULT false, -- broker-invited driver limited to s  
  approved_by_user_id            uuid REFERENCES identity.users(id),  
  approved_at                   timestamptz,  
  created_at                    timestamptz NOT NULL DEFAULT now(),  
  updated_at                    timestamptz NOT NULL DEFAULT now()
```

```

);
CREATE INDEX ix_drivers_org ON transport.drivers (organization_id, status);
CREATE INDEX ix_drivers_license_expiry ON transport.drivers (license_expiry) WHERE status = 'AC'

-- PT1: own fleet (vehicle + driver) or broker-assigned external carrier (D-20)
CREATE TABLE transport.trip_assignments (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL REFERENCES svc.orders(id),
  status            core.assignment_status NOT NULL DEFAULT 'ACTIVE',
  is_external        boolean NOT NULL DEFAULT false,
  vehicle_id        uuid REFERENCES transport.vehicles(id),
  driver_id         uuid REFERENCES transport.drivers(id),
  driver_user_id    uuid REFERENCES identity.users(id),  -- also set for invited guests
  external_company_name text,
  external_cr_number text,
  external_driver_name text,
  external_driver_phone text,
  external_plate_number text,
  external_vehicle_type_code text REFERENCES ref.vehicle_types(code),
  assigned_by_user_id uuid NOT NULL REFERENCES identity.users(id),
  assigned_at        timestampz NOT NULL DEFAULT now(),
  ended_at           timestampz,
  end_reason         text,                                -- REASSIGNED | CANCELLED | COMPLETED
  created_at         timestampz NOT NULL DEFAULT now(),
  updated_at         timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_assignment_shape CHECK (
    (NOT is_external AND vehicle_id IS NOT NULL AND driver_id IS NOT NULL) OR
    (is_external AND external_driver_name IS NOT NULL AND external_plate_number IS NOT NULL))
);
CREATE UNIQUE INDEX ux_assignments_active ON transport.trip_assignments (order_id) WHERE status = 'ACTIVE';
CREATE INDEX ix_assignments_driver ON transport.trip_assignments (driver_user_id, status);
CREATE INDEX ix_assignments_vehicle ON transport.trip_assignments (vehicle_id, status);
-- Overlap rule (one active trip per driver / vehicle at a time) is enforced by the application

-- Driver / provider trip events; client_event_id makes offline sync idempotent
CREATE TABLE transport.trip_events (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL REFERENCES svc.orders(id),
  assignment_id      uuid NOT NULL REFERENCES transport.trip_assignments(id),
  client_event_id    uuid NOT NULL UNIQUE,
  event_type         core.trip_event_type NOT NULL,
  occurred_at        timestampz NOT NULL,                -- device time (accepted up to 72 h late)
  received_at        timestampz NOT NULL DEFAULT now(),

```

```

source                core.update_source NOT NULL,
recorded_by_user_id   uuid NOT NULL REFERENCES identity.users(id),
unit_count            numeric(12,2),          -- CARGO_COLLECTED
border_point          text,                  -- BORDER_CROSSED
photo_file_ids        uuid[] NOT NULL DEFAULT '{}',
document_file_ids     uuid[] NOT NULL DEFAULT '{}',
location              geography(Point, 4326),
note                  text,
corrects_event_id     uuid REFERENCES transport.trip_events(id),  -- undo within 2 minutes
created_at            timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_trip_events_collect CHECK (event_type <> 'CARGO_COLLECTED' OR unit_count IS NOT
CONSTRAINT ck_trip_events_correction CHECK ((event_type = 'CORRECTION') = (corrects_event_id :
);

CREATE INDEX ix_trip_events_order ON transport.trip_events (order_id, occurred_at);

CREATE TABLE transport.trip_issues (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL REFERENCES svc.orders(id),
  assignment_id      uuid NOT NULL REFERENCES transport.trip_assignments(id),
  client_event_id    uuid NOT NULL UNIQUE,
  issue_type        core.trip_issue_type NOT NULL,
  reason            text NOT NULL,
  eta_impact_min     integer,
  photo_file_ids     uuid[] NOT NULL DEFAULT '{}',
  reported_by_user_id uuid NOT NULL REFERENCES identity.users(id),
  reported_at        timestampz NOT NULL,
  resolved_at        timestampz,
  created_at         timestampz NOT NULL DEFAULT now(),
  updated_at         timestampz NOT NULL DEFAULT now()
);

CREATE INDEX ix_trip_issues_open ON transport.trip_issues (order_id) WHERE resolved_at IS NULL;

-- High-volume GPS history, partitioned by month (§23). The latest position lives in Redis.
-- No foreign keys on purpose (write throughput); order_id is validated at ingest.
CREATE TABLE transport.tracking_points (
  id                uuid NOT NULL,
  order_id          uuid NOT NULL,
  assignment_id      uuid,
  recorded_at        timestampz NOT NULL,
  received_at        timestampz NOT NULL DEFAULT now(),
  location           geography(Point, 4326) NOT NULL,
  accuracy_m         real,
  speed_kph          real,

```

```
heading          smallint CHECK (heading IS NULL OR heading BETWEEN 0 AND 359),
source           core.update_source NOT NULL DEFAULT 'DRIVER',
is_downsampled   boolean NOT NULL DEFAULT false,
PRIMARY KEY (id, recorded_at)
) PARTITION BY RANGE (recorded_at);
CREATE INDEX ix_tracking_points_order ON transport.tracking_points (order_id, recorded_at DESC);
```

13. warehouse

```
CREATE TABLE warehouse.sites (  
  id                uuid PRIMARY KEY,  
  organization_id   uuid NOT NULL REFERENCES org.organizations(id),  
  name              text NOT NULL,  
  city_id           uuid NOT NULL REFERENCES ref.cities(id),  
  address_id        uuid REFERENCES org.addresses(id),  
  location           geography(Point, 4326),  
  storage_type_codes text[] NOT NULL DEFAULT '{}',    -- ref.storage_types  
  capacity_pallet_positions integer CHECK (capacity_pallet_positions IS NULL OR capacity_pallet_positions < 1000),  
  temperature_ranges jsonb,  
  working_hours      jsonb,  
  is_active           boolean NOT NULL DEFAULT true,  
  deleted_at          timestampz,  
  created_at          timestampz NOT NULL DEFAULT now(),  
  updated_at          timestampz NOT NULL DEFAULT now()  
);  
  
CREATE INDEX ix_sites_org ON warehouse.sites (organization_id) WHERE deleted_at IS NULL;  
CREATE INDEX ix_sites_city ON warehouse.sites (city_id) WHERE is_active AND deleted_at IS NULL;  
  
CREATE TABLE warehouse.storage_contracts (  
  id                uuid PRIMARY KEY,  
  order_id          uuid NOT NULL UNIQUE REFERENCES svc.orders(id),  
  site_id           uuid REFERENCES warehouse.sites(id),    -- set when the provider creates the contract  
  storage_type_code  text NOT NULL REFERENCES ref.storage_types(code),  
  start_date         date NOT NULL,  
  end_date           date NOT NULL,  
  booked_capacity_value numeric(10,2) NOT NULL,  
  booked_capacity_unit core.space_unit NOT NULL,  
  agreed_amount      core.halalas NOT NULL,  
  billing_mode        text NOT NULL DEFAULT 'UPFRONT' CHECK (billing_mode IN ('UPFRONT', 'MONTHLY', 'ANNUAL')),  
  status             core.storage_contract_status NOT NULL DEFAULT 'PENDING_INTAKE',  
  activated_at        timestampz,  
  closing_at         timestampz,  
  reconciled_at       timestampz,  
  closed_at          timestampz,  
  version            integer NOT NULL DEFAULT 1,  
  created_at          timestampz NOT NULL DEFAULT now(),  
  updated_at          timestampz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_contract_dates CHECK (end_date > start_date)  
);
```

```

CREATE INDEX ix_contracts_site ON warehouse.storage_contracts (site_id, status);
CREATE INDEX ix_contracts_ending ON warehouse.storage_contracts (end_date) WHERE status = 'ACTIVE';

CREATE TABLE warehouse.intake_slots (
    id                uuid PRIMARY KEY,
    contract_id       uuid NOT NULL REFERENCES warehouse.storage_contracts(id),
    slot_at           timestampz NOT NULL,
    expected_quantity numeric(12,2) NOT NULL CHECK (expected_quantity > 0),
    quantity_unit_code text NOT NULL REFERENCES ref.units_of_measure(code),
    transport_mode     core.warehouse_transport NOT NULL,
    linked_order_id   uuid REFERENCES svc.orders(id),           -- linked transport to the
    status            core.intake_slot_status NOT NULL DEFAULT 'PROPOSED',
    proposed_by_user_id uuid NOT NULL REFERENCES identity.users(id),
    confirmed_by_user_id uuid REFERENCES identity.users(id),
    confirmed_at       timestampz,
    created_at         timestampz NOT NULL DEFAULT now(),
    updated_at         timestampz NOT NULL DEFAULT now()
);

CREATE INDEX ix_intake_slots_contract ON warehouse.intake_slots (contract_id, slot_at);

-- Cached balances per item, updated in the same transaction as stock_movements (row lock).
-- A nightly job verifies them against the sum of movements.

CREATE TABLE warehouse.inventory_items (
    id                uuid PRIMARY KEY,
    contract_id       uuid NOT NULL REFERENCES warehouse.storage_contracts(id),
    description        text NOT NULL,                          -- "Packaging materials"
    commodity_code     text REFERENCES ref.commodity_categories(code),
    uom_code           text NOT NULL REFERENCES ref.units_of_measure(code),
    location_code      text,                                    -- "A-04"
    condition          text NOT NULL DEFAULT 'INTACT' CHECK (condition IN ('INTACT', 'DAMAGED', 'MIXED')),
    received_qty       numeric(12,2) NOT NULL DEFAULT 0,
    released_qty       numeric(12,2) NOT NULL DEFAULT 0,
    reserved_qty       numeric(12,2) NOT NULL DEFAULT 0,       -- approved releases not yet handed over
    adjusted_qty       numeric(12,2) NOT NULL DEFAULT 0,       -- signed sum of adjustments and returns
    available_qty      numeric(12,2) GENERATED ALWAYS AS (received_qty + adjusted_qty - released_qty),
    version            integer NOT NULL DEFAULT 1,
    created_at         timestampz NOT NULL DEFAULT now(),
    updated_at         timestampz NOT NULL DEFAULT now(),
    CONSTRAINT ck_inventory_non_negative CHECK (
        received_qty >= 0 AND released_qty >= 0 AND reserved_qty >= 0 AND
        received_qty + adjusted_qty - released_qty - reserved_qty >= 0)
);

CREATE INDEX ix_inventory_contract ON warehouse.inventory_items (contract_id);

```

-- Append-only stock ledger

```
CREATE TABLE warehouse.stock_movements (  
  id                uuid PRIMARY KEY,  
  item_id           uuid NOT NULL REFERENCES warehouse.inventory_items(id),  
  contract_id       uuid NOT NULL REFERENCES warehouse.storage_contracts(id),  
  movement_type     core.stock_movement_type NOT NULL,  
  quantity          numeric(12,2) NOT NULL,      -- signed: + in, - out  
  reason            text,  
  intake_slot_id    uuid REFERENCES warehouse.intake_slots(id),  
  release_request_id uuid,                        -- FK warehouse.release_requests ($21)  
  evidence_file_ids  uuid[] NOT NULL DEFAULT '{}',  
  source             core.update_source NOT NULL,  
  recorded_by_user_id  uuid REFERENCES identity.users(id),  
  recorded_by_staff_id  uuid REFERENCES identity.staff_users(id),  
  occurred_at        timestampz NOT NULL,  
  created_at         timestampz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_movement_sign CHECK (  
    (movement_type = 'INTAKE' AND quantity > 0) OR  
    (movement_type = 'RELEASE' AND quantity < 0) OR  
    (movement_type IN ('ADJUSTMENT', 'RETURN') AND quantity <> 0)),  
  CONSTRAINT ck_movement_adjustment_reason CHECK (movement_type <> 'ADJUSTMENT' OR reason IS NOT  
);  
  
CREATE INDEX ix_movements_item ON warehouse.stock_movements (item_id, occurred_at);  
CREATE INDEX ix_movements_contract ON warehouse.stock_movements (contract_id, occurred_at);
```

```
CREATE TABLE warehouse.release_requests (  
  id                uuid PRIMARY KEY,  
  reference          text NOT NULL UNIQUE,      -- LX-260148-R2  
  contract_id       uuid NOT NULL REFERENCES warehouse.storage_contracts(id),  
  order_id          uuid NOT NULL REFERENCES svc.orders(id),  
  sequence_no       integer NOT NULL,  
  release_date       date NOT NULL,  
  collection_method  core.collection_method NOT NULL,  
  linked_order_id    uuid REFERENCES svc.orders(id),      -- linked carrier  
  collector_name     text,                                -- self-collection  
  collector_phone    text,  
  collector_id_last4 text,  
  carrier_trip_reference text,  
  notes              text,  
  is_final           boolean NOT NULL DEFAULT false,      -- releases all remaining  
  status             core.release_status NOT NULL DEFAULT 'REQUESTED',  
  requested_by_user_id  uuid NOT NULL REFERENCES identity.users(id),
```

```

approved_by_user_id      uuid REFERENCES identity.users(id),
approved_at              timestamptz,
rejected_reason          text,
picking_at              timestamptz,
ready_at                timestamptz,
handed_over_at          timestamptz,
handover_evidence_file_ids uuid[] NOT NULL DEFAULT '{}',
completed_at            timestamptz,
cancelled_at            timestamptz,
version                 integer NOT NULL DEFAULT 1,
created_at              timestamptz NOT NULL DEFAULT now(),
updated_at              timestamptz NOT NULL DEFAULT now(),
UNIQUE (contract_id, sequence_no),
CONSTRAINT ck_release_reference CHECK (reference ~ '^[A-Z]{2,4}-[0-9]{3,}-R[0-9]+$'),
CONSTRAINT ck_release_collector CHECK (collection_method <> 'SELF' OR collector_name IS NOT NULL);
);

CREATE INDEX ix_releases_contract ON warehouse.release_requests (contract_id, status);
CREATE INDEX ix_releases_stuck ON warehouse.release_requests (approved_at) WHERE status IN ('API', 'STUCK');

CREATE TABLE warehouse.release_request_lines (
    id                uuid PRIMARY KEY,
    release_request_id uuid NOT NULL REFERENCES warehouse.release_requests(id) ON DELETE CASCADE,
    item_id           uuid NOT NULL REFERENCES warehouse.inventory_items(id),
    quantity          numeric(12,2) NOT NULL CHECK (quantity > 0),
    UNIQUE (release_request_id, item_id)
);

-- WH07 storage reconciliation (acceptance = RC equivalent for warehousing)
CREATE TABLE warehouse.reconciliations (
    id                uuid PRIMARY KEY,
    contract_id       uuid NOT NULL REFERENCES warehouse.storage_contracts(id),
    period_start      date NOT NULL,
    period_end        date NOT NULL,
    storage_days       integer NOT NULL CHECK (storage_days > 0),
    agreed_amount      core.halalas NOT NULL,
    additional_amount  core.halalas NOT NULL DEFAULT 0, -- only customer-approved additional amount
    remaining_qty      numeric(12,2) NOT NULL,
    status            core.reconciliation_status NOT NULL DEFAULT 'DRAFT',
    submitted_by_user_id uuid REFERENCES identity.users(id),
    submitted_at       timestamptz,
    accepted_by_user_id uuid REFERENCES identity.users(id),
    accepted_at        timestamptz,
    dispute_case_id    uuid, -- FK ops.cases (§21)

```

```
notes                text,
created_at           timestampz NOT NULL DEFAULT now(),
updated_at           timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_reconciliation_period CHECK (period_end >= period_start),
CONSTRAINT ck_reconciliation_closure CHECK (status <> 'ACCEPTED' OR remaining_qty = 0)
);
CREATE INDEX ix_reconciliations_contract ON warehouse.reconciliations (contract_id);
```

14. customs

```
-- CU05 / CU06 / PC2. An in-app authorization is not proof of Fasah activation: ACTIVE needs an
CREATE TABLE customs.broker_authorizations (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL REFERENCES svc.orders(id),
  customer_org_id   uuid NOT NULL REFERENCES org.organizations(id),
  broker_org_id     uuid NOT NULL REFERENCES org.organizations(id),
  authorizing_legal_name text NOT NULL,                -- snapshot from KYB
  authorizing_cr_number text,
  scope_text        text NOT NULL,                    -- "Order LX-2048 • defined validity"
  valid_from        date NOT NULL,
  valid_to          date NOT NULL,
  status            core.broker_authorization_status NOT NULL DEFAULT 'REQUESTED',
  official_reference text,
  evidence_file_id  uuid REFERENCES files.files(id),
  confirmed_source  core.update_source,                -- BROKER | ADMIN | INTEGRATION
  confirmed_by_user_id uuid REFERENCES identity.users(id),
  confirmed_by_staff_id uuid REFERENCES identity.staff_users(id),
  confirmed_at      timestampz,
  verified_by_staff_id uuid REFERENCES identity.staff_users(id), -- ops spot-check (D-08)
  verified_at       timestampz,
  revoked_at        timestampz,
  revoke_reason     text,
  rejected_reason   text,
  consent_id        uuid NOT NULL REFERENCES org.consent(id), -- BROKER_AUTHORIZATION co
  created_by_user_id uuid NOT NULL REFERENCES identity.users(id),
  version           integer NOT NULL DEFAULT 1,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_authorization_validity CHECK (valid_to >= valid_from),
  CONSTRAINT ck_authorization_active CHECK (status <> 'ACTIVE' OR (official_reference IS NOT I
);
-- At most one live authorization per order
CREATE UNIQUE INDEX ux_authorizations_live ON customs.broker_authorizations (order_id)
  WHERE status IN ('REQUESTED', 'PENDING_EXTERNAL_ACTION', 'ACTIVE');
CREATE INDEX ix_authorizations_unverified ON customs.broker_authorizations (confirmed_at)
  WHERE status = 'ACTIVE' AND verified_at IS NULL;

-- PC3 declaration execution (manual updates or an approved integration)
CREATE TABLE customs.declarations (
  id                uuid PRIMARY KEY,
```

```

order_id                uuid NOT NULL UNIQUE REFERENCES svc.orders(id),
declaration_reference    text,
manifest_reference       text,
status                  core.declaration_status NOT NULL DEFAULT 'NOT_STARTED',
duties_amount            core.halalas,          -- pass-through, separate from broker fe
duties_payer             core.duties_payer,
duties_receipt_document_id uuid REFERENCES files.documents(id),
inspection_notes         text,
evidence_file_ids       uuid[] NOT NULL DEFAULT '{}',
last_update_source       core.update_source,
last_updated_by_user_id  uuid REFERENCES identity.users(id),
last_updated_by_staff_id uuid REFERENCES identity.staff_users(id),
last_update_at           timestampz,
released_at              timestampz,
version                  integer NOT NULL DEFAULT 1,
created_at               timestampz NOT NULL DEFAULT now(),
updated_at               timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_declaration_submitted CHECK (status = 'NOT_STARTED' OR declaration_reference IS
);

CREATE INDEX ix_declarations_status ON customs.declarations (status, last_update_at);

-- Release notice = customs POD; CU09 port collection choice
CREATE TABLE customs.release_notices (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL UNIQUE REFERENCES svc.orders(id),
  document_id       uuid NOT NULL REFERENCES files.documents(id),
  released_at       timestampz NOT NULL,
  port_collection   core.port_collection,
  linked_order_id   uuid REFERENCES svc.orders(id), -- PORT_COLLECTION transport
  created_by_user_id uuid REFERENCES identity.users(id),
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now()
);

```

15. mkt : marketplace

```
CREATE TABLE mkt.product_categories (  
  id                uuid PRIMARY KEY,  
  parent_id         uuid REFERENCES mkt.product_categories(id),  
  slug              text NOT NULL UNIQUE,  
  name_ar           text NOT NULL,  
  name_en           text NOT NULL,  
  icon              text,  
  depth             smallint NOT NULL CHECK (depth BETWEEN 1 AND 3),  
  suggested_spec_keys jsonb NOT NULL DEFAULT '[]', -- e.g. ["dimensions","material","ply"]  
  is_active         boolean NOT NULL DEFAULT true,  
  sort_order        integer NOT NULL DEFAULT 0,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  updated_at        timestampz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_category_root CHECK ((parent_id IS NULL) = (depth = 1))  
);  
  
CREATE INDEX ix_categories_parent ON mkt.product_categories (parent_id, sort_order);  
  
CREATE TABLE mkt.products (  
  id                uuid PRIMARY KEY,  
  supplier_org_id   uuid NOT NULL REFERENCES org.organizations(id),  
  category_id       uuid NOT NULL REFERENCES mkt.product_categories(id),  
  name              text NOT NULL,  
  description        text,  
  specs             jsonb NOT NULL DEFAULT '[]', -- [{"key":"Material","value":"5-ply cartons"}]  
  origin_country_code char(2) NOT NULL REFERENCES ref.countries(code),  
  condition         core.product_condition NOT NULL DEFAULT 'NEW',  
  unit_code         text NOT NULL REFERENCES ref.units_of_measure(code),  
  pack_size         integer CHECK (pack_size IS NULL OR pack_size >= 1),  
  lead_time_days    smallint NOT NULL CHECK (lead_time_days BETWEEN 0 AND 90),  
  unit_price        core.halalas NOT NULL CHECK (unit_price > 0),  
  vat_treatment     core.vat_treatment NOT NULL DEFAULT 'EXCLUSIVE',  
  moq               integer NOT NULL DEFAULT 1 CHECK (moq >= 1),  
  stock_qty         integer NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),  
  reserved_qty      integer NOT NULL DEFAULT 0 CHECK (reserved_qty >= 0), -- 30-min checkout  
  low_stock_threshold integer,  
  city_id           uuid REFERENCES ref.cities(id),  
  supply_notes      text,  
  status            core.product_status NOT NULL DEFAULT 'DRAFT',  
  published_at      timestampz,  
  paused_at         timestampz,
```

```

removed_at          timestampz,
removal_reason_code text,
removed_by_staff_id uuid REFERENCES identity.staff_users(id),
listing_consent_id  uuid REFERENCES org.consent(id),    -- LISTING_ACCURACY
rating_avg          numeric(3,2),
rating_count        integer NOT NULL DEFAULT 0,
search_vector       tsvector,                        -- maintained by trigger ($22) with c
version             integer NOT NULL DEFAULT 1,
deleted_at          timestampz,
created_at          timestampz NOT NULL DEFAULT now(),
updated_at          timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_products_reserved CHECK (reserved_qty <= stock_qty),
CONSTRAINT ck_products_published CHECK (status NOT IN ('PUBLISHED', 'OUT_OF_STOCK', 'PAUSED'))
);

CREATE INDEX ix_products_supplier ON mkt.products (supplier_org_id, status) WHERE deleted_at IS
CREATE INDEX ix_products_browse ON mkt.products (category_id, unit_price) WHERE status = 'PUBLI
CREATE INDEX ix_products_city ON mkt.products (city_id) WHERE status = 'PUBLISHED';
CREATE INDEX ix_products_newest ON mkt.products (published_at DESC) WHERE status = 'PUBLISHED';
CREATE INDEX ix_products_search ON mkt.products USING gin (search_vector);
CREATE INDEX ix_products_name_trgm ON mkt.products USING gin (core.normalize_ar(name) gin_trgm_c

CREATE TABLE mkt.product_images (
  id          uuid PRIMARY KEY,
  product_id  uuid NOT NULL REFERENCES mkt.products(id) ON DELETE CASCADE,
  file_id     uuid NOT NULL REFERENCES files.files(id),
  sort_order  smallint NOT NULL DEFAULT 0,
  is_cover    boolean NOT NULL DEFAULT false,
  moderation_status text NOT NULL DEFAULT 'PENDING' CHECK (moderation_status IN ('PENDING', 'AI
  public_url  text,                                -- CDN URL after approval
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX ux_product_images_cover ON mkt.product_images (product_id) WHERE is_cover;

CREATE TABLE mkt.favorites (
  organization_id uuid NOT NULL REFERENCES org.organizations(id),
  user_id         uuid NOT NULL REFERENCES identity.users(id),
  product_id      uuid NOT NULL REFERENCES mkt.products(id),
  created_at      timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (organization_id, user_id, product_id)
);

-- One cart per buyer organization, grouped by supplier in the UI

```

[illegible]

```

CREATE TABLE mkt.purchase_orders (
  id                uuid PRIMARY KEY,
  reference          text NOT NULL UNIQUE,      -- PO-260083
  buyer_org_id      uuid NOT NULL REFERENCES org.organizations(id),
  buyer_user_id     uuid NOT NULL REFERENCES identity.users(id),
  supplier_org_id    uuid NOT NULL REFERENCES org.organizations(id),
  checkout_quote_id uuid NOT NULL UNIQUE REFERENCES mkt.checkout_quotes(id),
  status            core.po_status NOT NULL DEFAULT 'PENDING_PAYMENT',
  payment_status     core.payment_summary_status NOT NULL DEFAULT 'UNPAID',
  settlement_status  core.settlement_status,
  delivery_address_id uuid REFERENCES org.addresses(id),
  delivery_address_snapshot jsonb,
  shipping_method    core.shipping_method NOT NULL,
  linked_transport_order_id uuid REFERENCES svc.orders(id),
  products_subtotal  core.halalas NOT NULL,
  shipping_fee       core.halalas NOT NULL DEFAULT 0,
  other_fees         core.halalas NOT NULL DEFAULT 0,
  customs_amount     core.halalas NOT NULL DEFAULT 0,
  vat_amount         core.halalas NOT NULL,
  financing_cost     core.halalas NOT NULL DEFAULT 0,
  total_amount       core.halalas NOT NULL,
  commission_rule_id uuid NOT NULL,            -- FK fin.commission_rules (§21)
  commission_rate_bps core.bps NOT NULL,       -- 3.5% = 350
  commission_amount   core.halalas NOT NULL,
  commission_vat_amount core.halalas NOT NULL DEFAULT 0,
  net_to_supplier     core.halalas NOT NULL,
  purchase_terms_consent_id uuid NOT NULL REFERENCES org.consent(id),
  readiness_due_at    timestamptz,             -- placed + lead time (business days)
  placed_at          timestamptz,
  accepted_at        timestamptz,
  rejected_at        timestamptz,
  rejection_reason    text,
  preparing_at       timestamptz,
  ready_at           timestamptz,
  handed_over_at     timestamptz,
  delivered_at       timestamptz,
  received_at        timestamptz,
  return_window_ends_at timestamptz,          -- received + 3 days (D-11 / D-21)
  completed_at       timestamptz,
  cancelled_at       timestamptz,
  version            integer NOT NULL DEFAULT 1,
  created_at         timestamptz NOT NULL DEFAULT now(),

```

```

updated_at          timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_po_reference CHECK (reference ~ '^PO-[0-9]{6,}$'),
CONSTRAINT ck_po_total CHECK (total_amount = products_subtotal + shipping_fee + other_fees + (
    + vat_amount + financing_cost),
CONSTRAINT ck_po_parties CHECK (buyer_org_id <> supplier_org_id)
);
CREATE INDEX ix_po_buyer ON mkt.purchase_orders (buyer_org_id, created_at DESC);
CREATE INDEX ix_po_supplier ON mkt.purchase_orders (supplier_org_id, status, created_at DESC);
CREATE INDEX ix_po_readiness_due ON mkt.purchase_orders (readiness_due_at) WHERE status IN ('PLU
CREATE INDEX ix_po_return_window ON mkt.purchase_orders (return_window_ends_at) WHERE status =

-- Price, name and VAT treatment are frozen at purchase time
CREATE TABLE mkt.purchase_order_items (
    id                uuid PRIMARY KEY,
    purchase_order_id  uuid NOT NULL REFERENCES mkt.purchase_orders(id),
    product_id         uuid NOT NULL REFERENCES mkt.products(id),
    product_name_snapshot text NOT NULL,
    unit_code          text NOT NULL REFERENCES ref.units_of_measure(code),
    unit_price         core.halalas NOT NULL,
    vat_treatment      core.vat_treatment NOT NULL,
    vat_rate_bps       core.bps NOT NULL,
    quantity           integer NOT NULL CHECK (quantity > 0),
    line_subtotal      core.halalas NOT NULL,          -- excluding VAT
    line_vat           core.halalas NOT NULL,
    received_qty       integer,
    returned_qty       integer NOT NULL DEFAULT 0,
    created_at         timestampz NOT NULL DEFAULT now(),
    CONSTRAINT ck_po_items_returned CHECK (returned_qty >= 0 AND (received_qty IS NULL OR returned
);
CREATE INDEX ix_po_items_po ON mkt.purchase_order_items (purchase_order_id);

-- S06 shipment readiness
CREATE TABLE mkt.shipment_readiness (
    id                uuid PRIMARY KEY,
    purchase_order_id  uuid NOT NULL UNIQUE REFERENCES mkt.purchase_orders(id),
    prepared_qty       integer NOT NULL CHECK (prepared_qty > 0),
    checklist          jsonb NOT NULL,                -- {"fullQuantity":true,"specMatch":true
    packaging_photo_file_ids uuid[] NOT NULL DEFAULT '{}',
    sales_document_ids uuid[] NOT NULL DEFAULT '{}',   -- invoice & packing list (files.doc
    pickup_slot_at     timestampz,
    loading_instructions text,
    marked_ready_by_user_id uuid NOT NULL REFERENCES identity.users(id),
    marked_ready_at     timestampz NOT NULL DEFAULT now(),

```

```

    created_at          timestampz NOT NULL DEFAULT now(),
    updated_at          timestampz NOT NULL DEFAULT now()
);

```

-- S07 handover to carrier

```

CREATE TABLE mkt.po_handovers (
    id                  uuid PRIMARY KEY,
    purchase_order_id   uuid NOT NULL UNIQUE REFERENCES mkt.purchase_orders(id),
    linked_transport_order_id uuid REFERENCES svc.orders(id),
    unit_count          integer NOT NULL CHECK (unit_count > 0),
    photo_file_ids      uuid[] NOT NULL DEFAULT '{}',
    carrier_signature_file_id uuid REFERENCES files.files(id),
    picked_up_at        timestampz NOT NULL,
    recorded_by_user_id uuid NOT NULL REFERENCES identity.users(id),
    created_at          timestampz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE mkt.return_requests (
    id                  uuid PRIMARY KEY,
    reference           text NOT NULL UNIQUE,          -- RT-260012
    purchase_order_id   uuid NOT NULL REFERENCES mkt.purchase_orders(id),
    buyer_org_id        uuid NOT NULL REFERENCES org.organizations(id),
    supplier_org_id     uuid NOT NULL REFERENCES org.organizations(id),
    reason              core.return_reason NOT NULL,
    description         text NOT NULL,
    evidence_file_ids   uuid[] NOT NULL,
    preferred_resolution core.return_preference NOT NULL,
    status              core.return_status NOT NULL DEFAULT 'REQUESTED',
    window_ends_at      timestampz NOT NULL,          -- copied from the PO at creation
    supplier_decision_note text,
    info_requested_note text,
    escalated_at        timestampz,
    escalation_case_id  uuid,                          -- FK ops.cases ($21)
    admin_decision_note text,
    decided_by_staff_id uuid REFERENCES identity.staff_users(id),
    return_cost_bearer  core.cost_bearer,              -- SELLER for non-conforming goods (supp
    pickup_order_id     uuid REFERENCES svc.orders(id), -- RETURN_PICKUP transport
    returned_at         timestampz,
    resolution          core.return_resolution,
    refund_id           uuid,                          -- FK fin.refunds ($21)
    replacement_po_id   uuid REFERENCES mkt.purchase_orders(id),
    resolved_at         timestampz,
    closed_at           timestampz,

```

```

version                integer NOT NULL DEFAULT 1,
created_at             timestampz NOT NULL DEFAULT now(),
updated_at            timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_return_reference CHECK (reference ~ '^RT-[0-9]{6,}$'),
CONSTRAINT ck_return_window CHECK (created_at <= window_ends_at),
CONSTRAINT ck_return_evidence CHECK (cardinality(evidence_file_ids) > 0)
);

CREATE INDEX ix_returns_po ON mkt.return_requests (purchase_order_id);
CREATE INDEX ix_returns_supplier ON mkt.return_requests (supplier_org_id, status);
CREATE INDEX ix_returns_escalated ON mkt.return_requests (escalated_at) WHERE status = 'ESCALATED';

CREATE TABLE mkt.return_request_lines (
  id                uuid PRIMARY KEY,
  return_request_id uuid NOT NULL REFERENCES mkt.return_requests(id) ON DELETE CASCADE,
  po_item_id        uuid NOT NULL REFERENCES mkt.purchase_order_items(id),
  quantity          integer NOT NULL CHECK (quantity > 0),
  UNIQUE (return_request_id, po_item_id)
);

```

16. fin : payments, ledger, invoices, settlements

Amounts follow the rules in [modules/12-payments-finance.md](#). The invoicing model (D-03) and commission base (D-02) change *values*, not the table shapes.

16.1 Commission rules, payments and refunds

-- Versioned commission rules; orders and POs snapshot the rule id and rate at creation

```
CREATE TABLE fin.commission_rules (  
  id                uuid PRIMARY KEY,  
  category          core.commission_category NOT NULL,  
  rate_bps          core.bps NOT NULL,          -- 1000 = 10%, 2000 = 20%, 350 = 3.5%  
  vat_on_commission boolean NOT NULL DEFAULT true, -- D-02 default  
  vat_rate_bps      core.bps NOT NULL DEFAULT 1500,  
  base_definition   text NOT NULL,              -- e.g. "service fee + taxable charges,  
  effective_from    timestamptz NOT NULL,  
  effective_to      timestamptz,  
  justification     text,  
  created_by_staff_id uuid REFERENCES identity.staff_users(id),  
  approved_by_staff_id uuid REFERENCES identity.staff_users(id),  
  approved_at       timestamptz,  
  created_at        timestamptz NOT NULL DEFAULT now(),  
  updated_at        timestamptz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_commission_period CHECK (effective_to IS NULL OR effective_to > effective_from),  
  CONSTRAINT ck_commission_maker_checker CHECK (approved_by_staff_id IS NULL OR created_by_staff_id = approved_by_staff_id OR approved_by_staff_id <> created_by_staff_id),  
  -- No two approved rules of the same category may overlap in time  
  CONSTRAINT ex_commission_no_overlap EXCLUDE USING gist (  
    category WITH =, tstzrange(effective_from, effective_to, '[') WITH &&) WHERE (approved_at IS NOT NULL  
);
```

```
CREATE TABLE fin.payment_intents (  
  id                uuid PRIMARY KEY,  
  payable_type      core.payable_type NOT NULL,  
  order_id          uuid REFERENCES svc.orders(id),  
  purchase_order_id uuid REFERENCES mkt.purchase_orders(id),  
  additional_charge_id uuid REFERENCES svc.additional_charges(id),  
  insurance_request_id uuid,                -- FK ops.insurance_requests ($21)  
  payer_org_id      uuid NOT NULL REFERENCES org.organizations(id),  
  payer_user_id     uuid NOT NULL REFERENCES identity.users(id),  
  amount            core.halalas NOT NULL CHECK (amount > 0), -- always computed server-side  
  currency          core.currency_code NOT NULL,  
  method            core.payment_method NOT NULL,  
  gateway           text NOT NULL,          -- moyasar | hyperpay | tap | fake (D-02)  
  gateway_payment_id text,  
  status            core.payment_intent_status NOT NULL DEFAULT 'INITIATED',  
  failure_code      text,  
  failure_message   text,
```

```

client_payload      jsonb,                -- non-sensitive data for the SDK / host
expires_at         timestampz NOT NULL,    -- payment hold (30 min)
paid_at            timestampz,
refunded_amount     core.halalas NOT NULL DEFAULT 0,
reconciliation_status core.reconciliation_state NOT NULL DEFAULT 'PENDING',
reconciled_at       timestampz,
version            integer NOT NULL DEFAULT 1,
created_at          timestampz NOT NULL DEFAULT now(),
updated_at          timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_intent_payable CHECK (
    (payable_type = 'ORDER'                AND order_id IS NOT NULL
     AND num_nonnulls(purchase_order_id, additional_charge_id, insurance_request_id) = 0) OR
    (payable_type = 'PURCHASE_ORDER'        AND purchase_order_id IS NOT NULL
     AND num_nonnulls(order_id, additional_charge_id, insurance_request_id) = 0) OR
    (payable_type = 'ADDITIONAL_CHARGE' AND additional_charge_id IS NOT NULL
     AND num_nonnulls(order_id, purchase_order_id, insurance_request_id) = 0) OR
    (payable_type = 'INSURANCE_PREMIUM' AND insurance_request_id IS NOT NULL
     AND num_nonnulls(order_id, purchase_order_id, additional_charge_id) = 0)),
CONSTRAINT ck_intent_refunded CHECK (refunded_amount <= amount),
CONSTRAINT ck_intent_paid CHECK (status NOT IN ('PAID', 'PARTIALLY_REFUNDED', 'REFUNDED')
                                OR (paid_at IS NOT NULL AND gateway_payment_id IS NOT NULL))
);

CREATE UNIQUE INDEX ux_intents_gateway ON fin.payment_intents (gateway, gateway_payment_id) WHERE
-- At most one live or successful intent per payable (failed / expired intents can be retried)
CREATE UNIQUE INDEX ux_intents_order_live ON fin.payment_intents (order_id)
    WHERE order_id IS NOT NULL AND status IN ('INITIATED', 'PENDING', 'PAID', 'PARTIALLY_REFUNDED')
CREATE UNIQUE INDEX ux_intents_po_live ON fin.payment_intents (purchase_order_id)
    WHERE purchase_order_id IS NOT NULL AND status IN ('INITIATED', 'PENDING', 'PAID', 'PARTIALLY_REFUNDED')
CREATE UNIQUE INDEX ux_intents_charge_live ON fin.payment_intents (additional_charge_id)
    WHERE additional_charge_id IS NOT NULL AND status IN ('INITIATED', 'PENDING', 'PAID', 'PARTIALLY_REFUNDED')
CREATE UNIQUE INDEX ux_intents_insurance_live ON fin.payment_intents (insurance_request_id)
    WHERE insurance_request_id IS NOT NULL AND status IN ('INITIATED', 'PENDING', 'PAID', 'PARTIALLY_REFUNDED')
CREATE INDEX ix_intents_pending ON fin.payment_intents (created_at) WHERE status = 'PENDING';
CREATE INDEX ix_intents_unreconciled ON fin.payment_intents (paid_at) WHERE paid_at IS NOT NULL

-- Raw gateway webhooks (idempotent on the gateway event id)
CREATE TABLE fin.payment_events (
    id                uuid PRIMARY KEY,
    gateway            text NOT NULL,
    gateway_event_id  text NOT NULL,
    event_type         text NOT NULL,
    payment_intent_id  uuid REFERENCES fin.payment_intents(id),
    payload            jsonb NOT NULL,                -- no card data is ever received (hosted)

```

```

signature_valid    boolean NOT NULL,
received_at        timestampz NOT NULL DEFAULT now(),
processed_at       timestampz,
processing_error    text,
UNIQUE (gateway, gateway_event_id)
);
CREATE INDEX ix_payment_events_unprocessed ON fin.payment_events (received_at) WHERE processed_

CREATE TABLE fin.refunds (
  id                uuid PRIMARY KEY,
  payment_intent_id uuid NOT NULL REFERENCES fin.payment_intents(id),
  amount            core.halalas NOT NULL CHECK (amount > 0),
  reason            core.refund_reason NOT NULL,
  cancellation_request_id uuid,                -- FK ops.cancellation_requests (§21)
  case_id           uuid,                      -- FK ops.cases (§21)
  return_request_id uuid REFERENCES mkt.return_requests(id),
  status            core.refund_status NOT NULL DEFAULT 'REQUESTED',
  gateway_refund_id text,
  failure_reason     text,
  manual_bank_reference text,                -- fallback when the original method can't
  requested_by_user_id uuid REFERENCES identity.users(id),
  requested_by_staff_id uuid REFERENCES identity.staff_users(id),
  approved_by_staff_id uuid REFERENCES identity.staff_users(id), -- maker-checker above the line
  idempotency_key    text NOT NULL UNIQUE,
  requested_at       timestampz NOT NULL DEFAULT now(),
  processed_at       timestampz,
  completed_at       timestampz,
  created_at         timestampz NOT NULL DEFAULT now(),
  updated_at         timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_refund_maker_checker CHECK (approved_by_staff_id IS NULL OR requested_by_staff_id =
                                          OR approved_by_staff_id <> requested_by_staff_id)
);
CREATE INDEX ix_refunds_intent ON fin.refunds (payment_intent_id);
CREATE INDEX ix_refunds_open ON fin.refunds (status, requested_at)
  WHERE status IN ('REQUESTED', 'PROCESSING', 'FAILED', 'MANUAL_PENDING');

```

16.2 Invoices

-- Gap-free numbering per issuer and year: row-locked inside the invoicing transaction

```
CREATE TABLE fin.invoice_sequences (  
  issuer_key    text NOT NULL,                -- 'PLATFORM' or the issuer organization  
  year          smallint NOT NULL,  
  last_number   integer NOT NULL DEFAULT 0,  
  PRIMARY KEY (issuer_key, year)  
);
```

-- Immutable after issue (only ZATCA status fields change); corrections use credit notes

```
CREATE TABLE fin.invoices (  
  id            uuid PRIMARY KEY,  
  number        text NOT NULL,                -- INV-2026-000154 / CN-2026-000012  
  invoice_type   core.invoice_type NOT NULL,  
  issuer_type    text NOT NULL CHECK (issuer_type IN ('PLATFORM', 'ORGANIZATION')),  
  issuer_org_id  uuid REFERENCES org.organizations(id),  
  issuer_vat_number text,  
  recipient_org_id uuid REFERENCES org.organizations(id),  
  recipient_name text NOT NULL,  
  recipient_vat_number text,  
  order_id       uuid REFERENCES svc.orders(id),  
  purchase_order_id uuid REFERENCES mkt.purchase_orders(id),  
  original_invoice_id uuid REFERENCES fin.invoices(id), -- credit notes  
  issue_at       timestampz NOT NULL,  
  currency       core.currency_code NOT NULL,  
  subtotal       core.halalas NOT NULL,  
  vat_amount     core.halalas NOT NULL,  
  total          core.halalas NOT NULL,  
  zatca_uuid     uuid,  
  zatca_invoice_hash text,  
  zatca_previous_hash text,  
  zatca_qr       text,  
  zatca_status   core.zatca_status NOT NULL DEFAULT 'PENDING',  
  zatca_warnings jsonb,  
  zatca_submitted_at timestampz,  
  pdf_file_id    uuid REFERENCES files.files(id),  
  xml_file_id    uuid REFERENCES files.files(id),  
  created_at     timestampz NOT NULL DEFAULT now(),  
  updated_at     timestampz NOT NULL DEFAULT now(),  
  CONSTRAINT ux_invoices_number UNIQUE NULLS NOT DISTINCT (issuer_org_id, number),  
  CONSTRAINT ck_invoices_total CHECK (total = subtotal + vat_amount),  
  CONSTRAINT ck_invoices_issuer CHECK ((issuer_type = 'PLATFORM') = (issuer_org_id IS NULL));
```

```

CONSTRAINT ck_invoices_credit    CHECK ((invoice_type = 'CREDIT_NOTE') = (original_invoice_id = 0))
CONSTRAINT ck_invoices_subject   CHECK (num_nonnulls(order_id, purchase_order_id) <= 1)
);

CREATE INDEX ix_invoices_order ON fin.invoices (order_id) WHERE order_id IS NOT NULL;
CREATE INDEX ix_invoices_po ON fin.invoices (purchase_order_id) WHERE purchase_order_id IS NOT NULL;
CREATE INDEX ix_invoices_zatca_pending ON fin.invoices (issue_at) WHERE zatca_status IN ('PENDING', 'CANCELLED');

CREATE TABLE fin.invoice_lines (
    id                uuid PRIMARY KEY,
    invoice_id        uuid NOT NULL REFERENCES fin.invoices(id) ON DELETE CASCADE,
    line_no           smallint NOT NULL,
    description       text NOT NULL,
    quantity          numeric(12,2) NOT NULL DEFAULT 1,
    unit_price        core.halalas NOT NULL,
    line_subtotal     core.halalas NOT NULL,
    vat_rate_bps      core.bps NOT NULL,
    vat_amount        core.halalas NOT NULL,
    is_pass_through   boolean NOT NULL DEFAULT false,
    UNIQUE (invoice_id, line_no)
);

```

16.3 Settlements and payouts

```
CREATE TABLE fin.settlements (  
  id                uuid PRIMARY KEY,  
  reference         text NOT NULL UNIQUE,          -- STL-2600412  
  beneficiary_org_id  uuid NOT NULL REFERENCES org.organizations(id),  
  beneficiary_role   core.beneficiary_role NOT NULL,  
  order_id          uuid REFERENCES svc.orders(id),  
  purchase_order_id  uuid REFERENCES mkt.purchase_orders(id),  
  period_no         smallint NOT NULL DEFAULT 1,   -- > 1 only for monthly warehousing bill  
  commission_rule_id uuid NOT NULL REFERENCES fin.commission_rules(id),  
  gross_amount       core.halalas NOT NULL,        -- owed before commission (definition per  
  commission_rate_bps core.bps NOT NULL,  
  commission_amount  core.halalas NOT NULL,  
  commission_vat_amount core.halalas NOT NULL DEFAULT 0,  
  adjustments_total  core.signed_halalas NOT NULL DEFAULT 0,  
  net_amount         core.halalas NOT NULL,        -- never negative; excess deductions can  
  eligible_at        timestampz NOT NULL,          -- completion, or return-window close fo  
  payable_on         date NOT NULL,               -- eligible + 3 business days (ref.holic  
  status            core.settlement_status NOT NULL DEFAULT 'SCHEDULED',  
  hold_reason        text,  
  held_at           timestampz,  
  released_at       timestampz,  
  payout_id         uuid,                        -- FK fin.payouts ($21)  
  paid_at           timestampz,  
  failure_reason     text,  
  statement_file_id  uuid REFERENCES files.files(id),  
  version           integer NOT NULL DEFAULT 1,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  updated_at        timestampz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_settlement_reference CHECK (reference ~ '^STL-[0-9]{7,}$'),  
  CONSTRAINT ck_settlement_subject  CHECK (num_nonnulls(order_id, purchase_order_id) = 1),  
  CONSTRAINT ck_settlement_net      CHECK (net_amount = gross_amount - commission_amount - com  
  CONSTRAINT ck_settlement_hold     CHECK (status <> 'ON_HOLD' OR hold_reason IS NOT NULL),  
  CONSTRAINT ck_settlement_paid     CHECK (status <> 'PAID' OR (payout_id IS NOT NULL AND paid  
);  
  
CREATE UNIQUE INDEX ux_settlements_order ON fin.settlements (order_id, period_no) WHERE order_id  
CREATE UNIQUE INDEX ux_settlements_po ON fin.settlements (purchase_order_id) WHERE purchase_order_id  
CREATE INDEX ix_settlements_due ON fin.settlements (payable_on) WHERE status = 'SCHEDULED';  
CREATE INDEX ix_settlements_beneficiary ON fin.settlements (beneficiary_org_id, created_at DESC);
```

```
CREATE TABLE fin.settlement_adjustments (  
  id                uuid PRIMARY KEY,
```

```

settlement_id      uuid NOT NULL REFERENCES fin.settlements(id),
adjustment_type     core.adjustment_type NOT NULL,
amount             core.signed_halalas NOT NULL CHECK (amount <> 0),    -- negative = deduct:
reason             text NOT NULL,
case_id            uuid,                                                -- FK ops.cases ($21)
refund_id           uuid REFERENCES fin.refunds(id),
return_request_id   uuid REFERENCES mkt.return_requests(id),
created_by_staff_id uuid REFERENCES identity.staff_users(id),
created_at          timestampz NOT NULL DEFAULT now()
);

CREATE INDEX ix_settlement_adjustments ON fin.settlement_adjustments (settlement_id);

CREATE TABLE fin.payout_batches (
  id                uuid PRIMARY KEY,
  reference          text NOT NULL UNIQUE,                            -- PB-2026-0012
  status            core.payout_batch_status NOT NULL DEFAULT 'DRAFT',
  bank_format        text NOT NULL,                                    -- exporter key per bank
  total_amount       core.halalas NOT NULL DEFAULT 0,
  payout_count       integer NOT NULL DEFAULT 0,
  created_by_staff_id uuid NOT NULL REFERENCES identity.staff_users(id),
  submitted_at       timestampz,
  approved_by_staff_id uuid REFERENCES identity.staff_users(id),
  approved_at        timestampz,
  rejection_reason   text,
  exported_by_staff_id uuid REFERENCES identity.staff_users(id),
  exported_at        timestampz,
  export_file_id     uuid REFERENCES files.files(id),
  completed_at       timestampz,
  created_at         timestampz NOT NULL DEFAULT now(),
  updated_at         timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_batch_maker_checker CHECK (approved_by_staff_id IS NULL OR approved_by_staff_id = created_by_staff_id),
  CONSTRAINT ck_batch_export_approved CHECK (exported_at IS NULL OR approved_at IS NOT NULL)
);

-- One payout per beneficiary per batch; groups that beneficiary's due settlements
CREATE TABLE fin.payouts (
  id                uuid PRIMARY KEY,
  batch_id          uuid NOT NULL REFERENCES fin.payout_batches(id),
  reference          text NOT NULL UNIQUE,                            -- TRX-2602048 (sent as the bank payment)
  beneficiary_org_id uuid NOT NULL REFERENCES org.organizations(id),
  bank_account_id    uuid NOT NULL REFERENCES org.bank_accounts(id),
  iban_last4        char(4) NOT NULL,
  account_holder_snapshot text NOT NULL,

```

```

amount                core.halalas NOT NULL CHECK (amount > 0),
status                core.payout_status NOT NULL DEFAULT 'PENDING',
bank_transfer_reference text,
paid_at               timestampz,
failure_reason         text,
created_at            timestampz NOT NULL DEFAULT now(),
updated_at            timestampz NOT NULL DEFAULT now(),
UNIQUE (batch_id, beneficiary_org_id),
CONSTRAINT ck_payout_reference CHECK (reference ~ '^TRX-[0-9]{7,}$'),
CONSTRAINT ck_payout_paid      CHECK (status <> 'PAID' OR paid_at IS NOT NULL)
);
CREATE INDEX ix_payouts_beneficiary ON fin.payouts (beneficiary_org_id, created_at DESC);

```

16.4 Double-entry ledger

```
CREATE TABLE fin.ledger_accounts (  
  id                uuid PRIMARY KEY,  
  code              text NOT NULL,      -- gateway_clearing, customer_payments_held, provider_payable  
                                      -- commission_revenue, vat_output_payable, vat_collected_input  
                                      -- refunds_payable, cancellation_fees, payouts_in_transit  
  account_type      core.ledger_account_type NOT NULL,  
  normal_balance    core.entry_direction NOT NULL,  
  organization_id   uuid REFERENCES org.organizations(id),  -- set for per-organization payable  
  name              text NOT NULL,  
  currency          core.currency_code NOT NULL,  
  is_active         boolean NOT NULL DEFAULT true,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  CONSTRAINT ux_ledger_accounts_code UNIQUE NULLS NOT DISTINCT (code, organization_id)  
);
```

-- Append-only. Corrections are new transactions that reverse earlier ones.

```
CREATE TABLE fin.ledger_transactions (  
  id                uuid PRIMARY KEY,  
  kind              text NOT NULL,      -- PAYMENT_CAPTURED | ORDER_COMPLETED | COMMISSION  
                                      -- PAYOUT | PAYOUT_CONFIRMED | PAYOUT_FAILED | ADJUSTMENT  
  description       text,  
  occurred_at       timestampz NOT NULL,  
  order_id          uuid REFERENCES svc.orders(id),  
  purchase_order_id uuid REFERENCES mkt.purchase_orders(id),  
  payment_intent_id uuid REFERENCES fin.payment_intents(id),  
  refund_id         uuid REFERENCES fin.refunds(id),  
  settlement_id     uuid REFERENCES fin.settlements(id),  
  payout_id         uuid REFERENCES fin.payouts(id),  
  reverses_transaction_id uuid REFERENCES fin.ledger_transactions(id),  
  created_by_staff_id uuid REFERENCES identity.staff_users(id),  -- manual adjustments on  
  idempotency_key   text NOT NULL UNIQUE,      -- e.g. 'payment_captured:<intentId>  
  created_at        timestampz NOT NULL DEFAULT now()  
);
```

```
CREATE INDEX ix_ledger_tx_order ON fin.ledger_transactions (order_id) WHERE order_id IS NOT NULL  
CREATE INDEX ix_ledger_tx_po ON fin.ledger_transactions (purchase_order_id) WHERE purchase_order_id IS NOT NULL  
CREATE INDEX ix_ledger_tx_time ON fin.ledger_transactions (occurred_at);
```

```
CREATE TABLE fin.ledger_entries (  
  id                uuid PRIMARY KEY,  
  transaction_id    uuid NOT NULL REFERENCES fin.ledger_transactions(id),  
  account_id        uuid NOT NULL REFERENCES fin.ledger_accounts(id),
```

```
direction      core.entry_direction NOT NULL,
amount         core.halalas NOT NULL CHECK (amount > 0),
created_at     timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ix_ledger_entries_tx ON fin.ledger_entries (transaction_id);
CREATE INDEX ix_ledger_entries_account ON fin.ledger_entries (account_id, created_at);
--  $\Sigma$  debits =  $\Sigma$  credits per transaction is enforced by a deferred constraint trigger (§22)
```

17. ops : cancellations, cases, insurance

-- Cancellation policy as data (D-01). The engine picks the matching approved rule with the lowest priority

```
CREATE TABLE ops.cancellation_rules (  
  id                uuid PRIMARY KEY,  
  subject_type      text NOT NULL CHECK (subject_type IN ('ORDER', 'PURCHASE_ORDER')),  
  service_type      core.service_type,          -- NULL = any service  
  shipping_mode     core.shipping_mode,         -- e.g. sea/air bookings without a pickup  
  stage_code        text NOT NULL,              -- NOT_PAID | PAID_NOT_CONFIRMED | BEFORE_PICKUP  
  order_statuses    core.order_status[],  
  po_statuses       core.po_status[],  
  from_milestone_code text,                     -- rule applies from this milestone (inclusive)  
  until_milestone_code text,                   -- ... until this milestone (exclusive)  
  outcome           core.cancellation_outcome NOT NULL,  
  fee_bps           core.bps,  
  includes_return_costs boolean NOT NULL DEFAULT false,  
  explanation_key   text NOT NULL,              -- i18n key shown on X02 / X03  
  priority          smallint NOT NULL DEFAULT 100,  
  effective_from    timestamptz NOT NULL,  
  effective_to      timestamptz,  
  created_by_staff_id uuid REFERENCES identity.staff_users(id),  
  approved_by_staff_id uuid REFERENCES identity.staff_users(id),  
  approved_at       timestamptz,  
  created_at        timestamptz NOT NULL DEFAULT now(),  
  updated_at        timestamptz NOT NULL DEFAULT now(),  
  CONSTRAINT ck_cancel_rule_fee CHECK ((outcome = 'FEE_PERCENT') = (fee_bps IS NOT NULL)),  
  CONSTRAINT ck_cancel_rule_period CHECK (effective_to IS NULL OR effective_to > effective_from),  
  CONSTRAINT ck_cancel_rule_mc CHECK (approved_by_staff_id IS NULL OR created_by_staff_id IS NULL  
                                     OR approved_by_staff_id <> created_by_staff_id)  
);  
  
CREATE INDEX ix_cancel_rules_lookup ON ops.cancellation_rules (subject_type, service_type, priority);
```

```
CREATE TABLE ops.cases (  
  id                uuid PRIMARY KEY,  
  reference         text NOT NULL UNIQUE,       -- CASE-262048  
  case_type         core.case_type NOT NULL,  
  subject_type      core.case_subject_type NOT NULL,  
  subject_id        uuid,                      -- order / PO / return / payment intent  
  subject_reference text,                      -- LX-..., PO-..., RT-... for display and search  
  description       text,  
  evidence_file_ids uuid[] NOT NULL DEFAULT '{}',  
  opened_by_user_id uuid REFERENCES identity.users(id),
```

```

opened_by_staff_id        uuid REFERENCES identity.staff_users(id),
opened_by_org_id          uuid REFERENCES org.organizations(id),
opened_by_workspace       core.workspace_type,
counterparty_org_id       uuid REFERENCES org.organizations(id),
status                    core.case_status NOT NULL DEFAULT 'OPEN',
priority                  core.case_priority NOT NULL DEFAULT 'NORMAL',
assignee_staff_id         uuid REFERENCES identity.staff_users(id),
sla_first_response_due_at timestamptz,
sla_resolution_due_at     timestamptz,
first_responded_at        timestamptz,
sla_paused_at             timestamptz,          -- set while WAITING_ON_*
sla_paused_total          interval NOT NULL DEFAULT '0',
resolution_code            core.case_resolution_code,
resolution_note            text,
financial_outcome           jsonb,              -- {"refundId":..., "settlementAdjustment:
conversation_id            uuid,                -- FK msg.conversations (§21)
merged_into_case_id       uuid REFERENCES ops.cases(id),
resolved_at               timestamptz,
reopened_at               timestamptz,
closed_at                 timestamptz,
version                    integer NOT NULL DEFAULT 1,
created_at                 timestamptz NOT NULL DEFAULT now(),
updated_at                 timestamptz NOT NULL DEFAULT now(),
CONSTRAINT ck_case_reference CHECK (reference ~ '^CASE-[0-9]{6,}$'),
CONSTRAINT ck_case_opener  CHECK (num_nonnulls(opened_by_user_id, opened_by_staff_id) >= 1),
CONSTRAINT ck_case_subject CHECK ((subject_type = 'NONE') = (subject_id IS NULL)),
CONSTRAINT ck_case_resolved CHECK (status NOT IN ('RESOLVED', 'CLOSED') OR resolution_code IS
);

CREATE INDEX ix_cases_queue ON ops.cases (status, sla_resolution_due_at) WHERE status NOT IN ('I
CREATE INDEX ix_cases_assignee ON ops.cases (assignee_staff_id, status);
CREATE INDEX ix_cases_subject ON ops.cases (subject_type, subject_id);
CREATE INDEX ix_cases_opener_org ON ops.cases (opened_by_org_id, created_at DESC);

-- Staff-only notes (never visible to app users)
CREATE TABLE ops.case_internal_notes (
  id          uuid PRIMARY KEY,
  case_id     uuid NOT NULL REFERENCES ops.cases(id),
  staff_id    uuid NOT NULL REFERENCES identity.staff_users(id),
  body        text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX ix_case_notes_case ON ops.case_internal_notes (case_id, created_at);

```

```

CREATE TABLE ops.cancellation_requests (
  id                uuid PRIMARY KEY,
  order_id          uuid REFERENCES svc.orders(id),
  purchase_order_id uuid REFERENCES mkt.purchase_orders(id),
  requester         core.cancellation_requester NOT NULL,
  requested_by_user_id  uuid REFERENCES identity.users(id),
  requested_by_staff_id  uuid REFERENCES identity.staff_users(id),
  requested_by_org_id   uuid REFERENCES org.organizations(id),
  reason_code        text NOT NULL,          -- ref.reason_codes (domain CANCELLATION_REASON_CODE)
  note              text,
  status_at_request   text NOT NULL,          -- order / PO status snapshot
  milestone_at_request text,
  rule_id            uuid REFERENCES ops.cancellation_rules(id),
  engine_outcome      core.cancellation_outcome NOT NULL,
  engine_fee_bps      core.bps,
  expected_fee_amount core.halalas,          -- shown on X02 only when the outcome is 'REJECTED'
  status             core.cancellation_status NOT NULL DEFAULT 'REQUESTED',
  decided_fee_amount  core.halalas,
  refund_amount       core.halalas,
  fee_to_provider_amount core.halalas,       -- D-14 split
  fee_to_platform_amount core.halalas,
  decision_reason     text,                  -- shown to the customer on X03 / X04
  decided_by_staff_id  uuid REFERENCES identity.staff_users(id),
  approved_by_staff_id  uuid REFERENCES identity.staff_users(id), -- checker above threshold
  decided_at          timestamptz,
  case_id             uuid REFERENCES ops.cases(id),
  resume_status       text,                  -- restored if rejected
  version             integer NOT NULL DEFAULT 1,
  created_at          timestamptz NOT NULL DEFAULT now(),
  updated_at          timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT ck_cancel_subject CHECK (num_nonnulls(order_id, purchase_order_id) = 1),
  CONSTRAINT ck_cancel_requester CHECK (num_nonnulls(requested_by_user_id, requested_by_staff_id) = 1),
  CONSTRAINT ck_cancel_split CHECK (decided_fee_amount IS NULL OR fee_to_provider_amount IS NULL
                                     OR decided_fee_amount = fee_to_provider_amount + fee_to_platform_amount),
  CONSTRAINT ck_cancel_mc CHECK (approved_by_staff_id IS NULL OR decided_by_staff_id IS NULL
                                  OR approved_by_staff_id <> decided_by_staff_id),
  CONSTRAINT ck_cancel_review CHECK (engine_outcome <> 'REQUIRES_REVIEW' OR status <> 'AUTO_CANCELLED'),
);

CREATE UNIQUE INDEX ux_cancel_open_order ON ops.cancellation_requests (order_id)
  WHERE order_id IS NOT NULL AND status NOT IN ('REJECTED', 'COMPLETED');
CREATE UNIQUE INDEX ux_cancel_open_po ON ops.cancellation_requests (purchase_order_id)
  WHERE purchase_order_id IS NOT NULL AND status NOT IN ('REJECTED', 'COMPLETED');
CREATE INDEX ix_cancel_review_queue ON ops.cancellation_requests (created_at) WHERE status = 'UI_PENDING';

```

```

CREATE TABLE ops.insurers (
  id                uuid PRIMARY KEY,
  name_ar           text NOT NULL,
  name_en           text NOT NULL,
  coverage_modes    text[] NOT NULL,                -- SEA, ROAD, AIR, EXPRESS
  summary_ar        text,
  summary_en        text,
  terms_document_id uuid REFERENCES files.documents(id),
  is_contracted     boolean NOT NULL DEFAULT false,  -- false → IN1 shows "Illustrative name:
  contact_internal  jsonb,
  is_active         boolean NOT NULL DEFAULT true,
  sort_order        integer NOT NULL DEFAULT 0,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_insurer_modes CHECK (coverage_modes <@ ARRAY['SEA', 'ROAD', 'AIR', 'EXPRESS']::t
);

```

```

CREATE TABLE ops.insurance_requests (
  id                uuid PRIMARY KEY,
  order_id          uuid NOT NULL REFERENCES svc.orders(id),
  customer_org_id   uuid NOT NULL REFERENCES org.organizations(id),
  insurer_id        uuid NOT NULL REFERENCES ops.insurers(id),
  declared_value    core.halalas NOT NULL CHECK (declared_value > 0),
  currency          core.currency_code NOT NULL,
  route_snapshot    jsonb NOT NULL,
  commodity_snapshot jsonb NOT NULL,
  status            core.insurance_request_status NOT NULL DEFAULT 'REQUESTED',
  insurer_reference text,
  premium_amount    core.halalas,
  deductible_amount core.halalas,
  coverage_limit_amount core.halalas,
  exclusions_summary text,
  quote_document_id uuid REFERENCES files.documents(id),
  quote_valid_until timestampz,
  payment_route      core.insurance_payment_route,  -- D-07
  payment_proof_file_id uuid REFERENCES files.files(id),
  payment_confirmed_by_staff_id uuid REFERENCES identity.staff_users(id),
  consent_id         uuid NOT NULL REFERENCES org.consent(id),  -- INSURANCE_TERM:
  assigned_staff_id  uuid REFERENCES identity.staff_users(id),
  declined_reason    text,
  created_by_user_id uuid NOT NULL REFERENCES identity.users(id),
  version            integer NOT NULL DEFAULT 1,

```

```

        created_at            timestampz NOT NULL DEFAULT now(),
        updated_at            timestampz NOT NULL DEFAULT now(),
        CONSTRAINT ck_insurance_quoted CHECK (status NOT IN ('QUOTED', 'ACCEPTED', 'PAID', 'ISSUED')
                                                OR (premium_amount IS NOT NULL AND quote_valid_until IS
);
CREATE UNIQUE INDEX ux_insurance_live ON ops.insurance_requests (order_id) WHERE status NOT IN ('QUOTED', 'ACCEPTED', 'PAID', 'ISSUED');
CREATE INDEX ix_insurance_queue ON ops.insurance_requests (status, created_at);

-- Coverage exists only once a policy row exists (no "insured" wording before issuance)
CREATE TABLE ops.insurance_policies (
    id                uuid PRIMARY KEY,
    request_id        uuid NOT NULL UNIQUE REFERENCES ops.insurance_requests(id),
    policy_number      text NOT NULL,
    document_id        uuid NOT NULL REFERENCES files.documents(id),
    coverage_start      timestampz NOT NULL,
    coverage_end        timestampz NOT NULL,
    issued_at          timestampz NOT NULL DEFAULT now(),
    issued_by_staff_id uuid REFERENCES identity.staff_users(id),
    created_at          timestampz NOT NULL DEFAULT now(),
    updated_at          timestampz NOT NULL DEFAULT now(),
    CONSTRAINT ck_policy_period CHECK (coverage_end > coverage_start)
);

CREATE TABLE ops.insurance_claims (
    id                uuid PRIMARY KEY,
    policy_id          uuid NOT NULL REFERENCES ops.insurance_policies(id),
    incident_date       date NOT NULL,
    description         text NOT NULL,
    evidence_file_ids   uuid[] NOT NULL,
    status              core.insurance_claim_status NOT NULL DEFAULT 'SUBMITTED',
    amount_claimed      core.halalas,
    amount_approved     core.halalas,
    insurer_decision_note text, -- the insurer decides coverage; staff (
    decision_document_id uuid REFERENCES files.documents(id),
    decided_at          timestampz,
    paid_at             timestampz,
    case_id             uuid REFERENCES ops.cases(id),
    submitted_by_user_id uuid NOT NULL REFERENCES identity.users(id),
    created_at          timestampz NOT NULL DEFAULT now(),
    updated_at          timestampz NOT NULL DEFAULT now(),
    CONSTRAINT ck_claim_evidence CHECK (cardinality(evidence_file_ids) > 0)
);
CREATE INDEX ix_claims_policy ON ops.insurance_claims (policy_id);

```

18. msg : conversations and notifications

```
CREATE TABLE msg.conversations (  
  id                uuid PRIMARY KEY,  
  conversation_type core.conversation_type NOT NULL,  
  dedupe_key        text NOT NULL UNIQUE,      -- 'ORDER:<orderId>' | 'REQ:<requestId>:<provider>  
  context_type      text,                      -- ORDER | SERVICE_REQUEST | PRODUCT | PURCHASE_  
  context_id        uuid,  
  context_reference text,  
  title             text,  
  status            core.conversation_status NOT NULL DEFAULT 'OPEN',  
  pre_award_masking boolean NOT NULL DEFAULT true, -- contact details masked until award / f:  
  last_message_at   timestampz,  
  created_at        timestampz NOT NULL DEFAULT now(),  
  updated_at        timestampz NOT NULL DEFAULT now()  
);
```

```
CREATE INDEX ix_conversations_context ON msg.conversations (context_type, context_id);
```

```
CREATE TABLE msg.conversation_participants (  
  id                uuid PRIMARY KEY,  
  conversation_id    uuid NOT NULL REFERENCES msg.conversations(id) ON DELETE CASCADE,  
  organization_id    uuid REFERENCES org.organizations(id),      -- NULL for staff  
  user_id            uuid REFERENCES identity.users(id),          -- NULL = every member of the  
  staff_id           uuid REFERENCES identity.staff_users(id),  
  role               core.participant_role NOT NULL,  
  last_read_message_id uuid,  
  last_read_at       timestampz,  
  muted              boolean NOT NULL DEFAULT false,  
  joined_at          timestampz NOT NULL DEFAULT now(),  
  left_at            timestampz,  
  CONSTRAINT ck_participant_identity CHECK (num_nonnulls(organization_id, staff_id) = 1),  
  CONSTRAINT ux_participants UNIQUE NULLS NOT DISTINCT (conversation_id, organization_id, user_  
);
```

```
CREATE INDEX ix_participants_org ON msg.conversation_participants (organization_id) WHERE left_at IS NOT NULL;
```

```
CREATE INDEX ix_participants_user ON msg.conversation_participants (user_id) WHERE left_at IS NOT NULL;
```

```
CREATE TABLE msg.messages (  
  id                uuid PRIMARY KEY,  
  conversation_id    uuid NOT NULL REFERENCES msg.conversations(id),  
  kind              core.message_kind NOT NULL,  
  sender_user_id     uuid REFERENCES identity.users(id),  
  sender_org_id      uuid REFERENCES org.organizations(id),
```



```

priority          core.notification_priority NOT NULL DEFAULT 'NORMAL',
read_at           timestampz,
created_at        timestampz NOT NULL DEFAULT now(),
PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);
CREATE INDEX ix_notifications_user ON msg.notifications (user_id, created_at DESC);
CREATE INDEX ix_notifications_unread ON msg.notifications (user_id) WHERE read_at IS NULL;

-- Per-channel delivery tracking (push, SMS incl. OTP, email)
CREATE TABLE msg.notification_deliveries (
  id                uuid PRIMARY KEY,
  notification_id    uuid,                      -- NULL for OTP and direct sends
  user_id           uuid REFERENCES identity.users(id),
  channel           core.notification_channel NOT NULL,
  template_key      text,
  status            core.delivery_status NOT NULL DEFAULT 'QUEUED',
  provider          text,
  provider_message_id text,
  error             text,
  sent_at           timestampz,
  delivered_at      timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ix_deliveries_provider_msg ON msg.notification_deliveries (provider, provider_message_id);
CREATE INDEX ix_deliveries_failed ON msg.notification_deliveries (created_at) WHERE status = 'FAILED';

-- SECURITY and transactional categories cannot be disabled (enforced by the application)
CREATE TABLE msg.notification_preferences (
  user_id          uuid NOT NULL REFERENCES identity.users(id),
  category         core.notification_category NOT NULL,
  push_enabled     boolean NOT NULL DEFAULT true,
  sms_enabled      boolean NOT NULL DEFAULT true,
  email_enabled    boolean NOT NULL DEFAULT true,
  quiet_hours_start time,
  quiet_hours_end  time,
  updated_at       timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, category)
);

CREATE TABLE msg.notification_templates (
  id                uuid PRIMARY KEY,
  key               text NOT NULL,              -- e.g. 'quote.submitted'

```

```

channel          core.notification_channel NOT NULL,
locale           core.app_locale NOT NULL,
title            text,
body             text NOT NULL,
variables         text[] NOT NULL DEFAULT '{}',
is_transactional boolean NOT NULL DEFAULT false,    -- changes need maker-checker
version          integer NOT NULL DEFAULT 1,
updated_by_staff_id uuid REFERENCES identity.staff_users(id),
created_at       timestampz NOT NULL DEFAULT now(),
updated_at       timestampz NOT NULL DEFAULT now(),
UNIQUE (key, channel, locale)
);

```

19. audit

```

-- Append-only, partitioned by month (§23). Sensitive fields are redacted in before/after.
CREATE TABLE audit.audit_logs (
  id          uuid NOT NULL,
  occurred_at timestampz NOT NULL DEFAULT now(),
  actor_type  core.actor_type NOT NULL,
  actor_user_id  uuid,
  actor_staff_id uuid,
  actor_org_id  uuid,
  action        text NOT NULL,          -- e.g. 'order.milestone.corrected', 'pii.reveal'
  entity_type   text NOT NULL,
  entity_id     uuid,
  entity_reference text,                -- LX-..., PO-..., CASE-...
  before        jsonb,
  after         jsonb,
  reason        text,                  -- required for sensitive staff actions
  ip            inet,
  user_agent    text,
  request_id    text,
  PRIMARY KEY (id, occurred_at)
) PARTITION BY RANGE (occurred_at);
CREATE INDEX ix_audit_entity ON audit.audit_logs (entity_type, entity_id, occurred_at DESC);
CREATE INDEX ix_audit_reference ON audit.audit_logs (entity_reference) WHERE entity_reference IS NOT NULL;
CREATE INDEX ix_audit_staff ON audit.audit_logs (actor_staff_id, occurred_at DESC) WHERE actor_staff_id IS NOT NULL;
CREATE INDEX ix_audit_action ON audit.audit_logs (action, occurred_at DESC);

```

20. sys and cms

```
-- Transactional outbox: written in the same transaction as business rows, relayed to BullMQ
CREATE TABLE sys.outbox_events (
  id          uuid PRIMARY KEY,
  aggregate_type text NOT NULL,
  aggregate_id uuid NOT NULL,
  event_type   text NOT NULL,          -- e.g. 'order.completed'
  event_version smallint NOT NULL DEFAULT 1,
  payload      jsonb NOT NULL,
  occurred_at   timestampz NOT NULL DEFAULT now(),
  published_at  timestampz,
  attempts      integer NOT NULL DEFAULT 0,
  last_error    text
);

CREATE INDEX ix_outbox_unpublished ON sys.outbox_events (occurred_at) WHERE published_at IS NULL;

-- Handler-level idempotency for event consumers
CREATE TABLE sys.processed_events (
  event_id      uuid NOT NULL,
  handler       text NOT NULL,
  processed_at  timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (event_id, handler)
);

-- Idempotency-Key storage for money / order-creating POSTs (24 h)
CREATE TABLE sys.idempotency_keys (
  key          text NOT NULL,
  principal_id uuid NOT NULL,          -- user or staff id
  route        text NOT NULL,
  request_hash  bytea NOT NULL,
  response_status smallint,
  response_body jsonb,
  locked_until  timestampz,           -- in-flight protection
  created_at    timestampz NOT NULL DEFAULT now(),
  expires_at    timestampz NOT NULL,
  PRIMARY KEY (key, principal_id)
);

CREATE INDEX ix_idempotency_expiry ON sys.idempotency_keys (expires_at);

-- Human reference counters: LX, PO, RT, CASE, STL, TRX, PB per year (row-locked on increment)
CREATE TABLE sys.reference_counters (
```

```

prefix      text NOT NULL,
year        smallint NOT NULL,
last_value  bigint NOT NULL DEFAULT 0,
PRIMARY KEY (prefix, year)
);

```

```

CREATE TABLE sys.feature_flags (
  key          text PRIMARY KEY,          -- service.customs.enabled, marketplace.checkout
  description   text,
  enabled       boolean NOT NULL DEFAULT false,
  rules         jsonb NOT NULL DEFAULT '{}', -- scopes: environment, workspace, city, country
  is_kill_switch boolean NOT NULL DEFAULT false,
  owner         text,
  updated_by_staff_id uuid REFERENCES identity.staff_users(id),
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now()
);

```

-- Min app versions, maintenance mode, support contacts, upload limits, calendar.weekend, threshold

```

CREATE TABLE sys.app_config (
  key          text PRIMARY KEY,
  value        jsonb NOT NULL,
  description   text,
  updated_by_staff_id uuid REFERENCES identity.staff_users(id),
  created_at    timestampz NOT NULL DEFAULT now(),
  updated_at    timestampz NOT NULL DEFAULT now()
);

```

-- Maker-checker queue (payout batches, big refunds, rule changes, force-complete, broadcasts, etc)

```

CREATE TABLE sys.pending_approvals (
  id           uuid PRIMARY KEY,
  action_type  text NOT NULL,
  subject_type text NOT NULL,
  subject_id   uuid,
  summary      text NOT NULL,
  payload      jsonb NOT NULL,          -- what will be executed on approval
  amount       core.halalas,
  maker_staff_id uuid NOT NULL REFERENCES identity.staff_users(id),
  maker_reason text NOT NULL,
  status       core.approval_status NOT NULL DEFAULT 'PENDING',
  checker_staff_id uuid REFERENCES identity.staff_users(id),
  decision_note text,
  decided_at   timestampz,

```

```

expires_at      timestampz NOT NULL,          -- 72 h
created_at      timestampz NOT NULL DEFAULT now(),
updated_at      timestampz NOT NULL DEFAULT now(),
CONSTRAINT ck_approvals_distinct CHECK (checker_staff_id IS NULL OR checker_staff_id <> maker_
CONSTRAINT ck_approvals_decided  CHECK (status = 'PENDING' OR status = 'EXPIRED' OR checker_s
);
CREATE INDEX ix_approvals_pending ON sys.pending_approvals (created_at) WHERE status = 'PENDING

```

```

-- Inbound non-payment webhooks and integration callbacks (SMS DLR, Fasah, ZATCA, ...)

```

```

CREATE TABLE sys.integration_events (
  id          uuid PRIMARY KEY,
  integration  text NOT NULL,
  external_event_id text NOT NULL,
  event_type   text NOT NULL,
  payload      jsonb NOT NULL,
  signature_valid boolean,
  received_at  timestampz NOT NULL DEFAULT now(),
  processed_at timestampz,
  processing_error text,
  UNIQUE (integration, external_event_id)
);

```

```

CREATE TABLE sys.export_jobs (
  id          uuid PRIMARY KEY,
  staff_id    uuid NOT NULL REFERENCES identity.staff_users(id),
  report_key   text NOT NULL,
  filters      jsonb NOT NULL DEFAULT '{}',
  include_pii  boolean NOT NULL DEFAULT false,
  pii_reason    text,
  status       core.export_status NOT NULL DEFAULT 'QUEUED',
  row_count    integer,
  file_id      uuid REFERENCES files.files(id),
  error        text,
  created_at   timestampz NOT NULL DEFAULT now(),
  completed_at timestampz,
  expires_at   timestampz,          -- download link valid 24 h
  CONSTRAINT ck_export_pii_reason CHECK (NOT include_pii OR pii_reason IS NOT NULL)
);

```

```

CREATE TABLE sys.saved_views (
  id          uuid PRIMARY KEY,
  staff_id    uuid NOT NULL REFERENCES identity.staff_users(id) ON DELETE CASCADE,
  page_key    text NOT NULL,          -- 'orders', 'kyb-queue', ...

```

```

name          text NOT NULL,
filters       jsonb NOT NULL DEFAULT '{}',
columns       jsonb NOT NULL DEFAULT '[]',
is_default    boolean NOT NULL DEFAULT false,
created_at    timestampz NOT NULL DEFAULT now(),
updated_at    timestampz NOT NULL DEFAULT now(),
UNIQUE (staff_id, page_key, name)
);

```

```

CREATE TABLE cms.faqs (
  id                uuid PRIMARY KEY,
  category_key      text NOT NULL,
  question_ar       text NOT NULL,
  question_en       text NOT NULL,
  answer_ar         text NOT NULL,
  answer_en         text NOT NULL,
  workspaces        core.workspace_type[] NOT NULL DEFAULT '{CUSTOMER,SUPPLIER,PROVIDER,DRIVER}',
  sort_order        integer NOT NULL DEFAULT 0,
  is_active         boolean NOT NULL DEFAULT true,
  updated_by_staff_id uuid REFERENCES identity.staff_users(id),
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now()
);

```

```

CREATE TABLE cms.banners (
  id                uuid PRIMARY KEY,
  title_ar          text NOT NULL,
  title_en          text NOT NULL,
  body_ar           text,
  body_en           text,
  image_file_id     uuid REFERENCES files.files(id),
  deep_link         text,
  workspaces        core.workspace_type[] NOT NULL,
  city_ids          uuid[] NOT NULL DEFAULT '{}',
  service_types     core.service_type[] NOT NULL DEFAULT '{}',
  starts_at         timestampz NOT NULL,
  ends_at           timestampz,
  priority          smallint NOT NULL DEFAULT 100,
  is_dismissible    boolean NOT NULL DEFAULT true,
  is_active         boolean NOT NULL DEFAULT true,
  created_by_staff_id uuid REFERENCES identity.staff_users(id),
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
);

```

```

    CONSTRAINT ck_banner_period CHECK (ends_at IS NULL OR ends_at > starts_at)
);

CREATE TABLE cms.broadcasts (
    id                uuid PRIMARY KEY,
    title_ar          text NOT NULL,
    title_en          text NOT NULL,
    body_ar           text NOT NULL,
    body_en           text NOT NULL,
    deep_link         text,
    segment           jsonb NOT NULL,          -- workspace, city, activity, last active, has c
    channels           core.notification_channel[] NOT NULL,
    is_marketing       boolean NOT NULL DEFAULT false,  -- respects opt-outs; never SMS without
    scheduled_at       timestampz,
    status             core.broadcast_status NOT NULL DEFAULT 'DRAFT',
    estimated_reach    integer,
    sent_count         integer NOT NULL DEFAULT 0,
    approval_id        uuid REFERENCES sys.pending_approvals(id),
    created_by_staff_id uuid NOT NULL REFERENCES identity.staff_users(id),
    created_at         timestampz NOT NULL DEFAULT now(),
    updated_at         timestampz NOT NULL DEFAULT now()
);

```

21. Deferred foreign keys

These references point to tables defined in later sections. They are added after all tables exist.

```

ALTER TABLE org.licenses      ADD CONSTRAINT fk_licenses_document FOREIGN KEY (document_id) REFERENCES files.documents(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE org.bank_accounts ADD CONSTRAINT fk_bank_proof_document FOREIGN KEY (proof_document_id) REFERENCES files.documents(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE org.terms_documents ADD CONSTRAINT fk_terms_signoff_file FOREIGN KEY (legal_signoff_file_id) REFERENCES files.documents(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE files.documents    ADD CONSTRAINT fk_documents_current_version FOREIGN KEY (current_version_id) REFERENCES files.document_versions(id) DEFERRABLE INITIALLY DEFERRED; -- document and first version

ALTER TABLE svc.service_requests ADD CONSTRAINT fk_requests_parent_order FOREIGN KEY (parent_order_id) REFERENCES svc.orders(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE svc.orders           ADD CONSTRAINT fk_orders_commission_rule FOREIGN KEY (commission_rule_id) REFERENCES svc.order_links(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE svc.order_links      ADD CONSTRAINT fk_links_parent_po FOREIGN KEY (parent_purchase_order_id) REFERENCES svc.orders(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE svc.order_links      ADD CONSTRAINT fk_links_child_po FOREIGN KEY (child_purchase_order_id) REFERENCES svc.orders(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE svc.order_links      ADD CONSTRAINT fk_links_child_insurance FOREIGN KEY (child_insurance_id) REFERENCES svc.receipt_confirmations(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE svc.receipt_confirmations ADD CONSTRAINT fk_rc_po FOREIGN KEY (purchase_order_id) REFERENCES svc.orders(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE svc.receipt_confirmations ADD CONSTRAINT fk_rc_release FOREIGN KEY (release_id) REFERENCES svc.ratings(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE svc.ratings          ADD CONSTRAINT fk_ratings_po FOREIGN KEY (purchase_order_id) REFERENCES svc.orders(id) DEFERRABLE INITIALLY DEFERRED;

ALTER TABLE warehouse.stock_movements ADD CONSTRAINT fk_movements_release FOREIGN KEY (release_id) REFERENCES warehouse.reconciliations(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE warehouse.reconciliations ADD CONSTRAINT fk_reconciliation_case FOREIGN KEY (dispute_id) REFERENCES warehouse.stock_movements(id) DEFERRABLE INITIALLY DEFERRED;

ALTER TABLE mkt.purchase_orders ADD CONSTRAINT fk_po_commission_rule FOREIGN KEY (commission_rule_id) REFERENCES mkt.return_requests(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE mkt.return_requests ADD CONSTRAINT fk_returns_case FOREIGN KEY (escalation_id) REFERENCES mkt.return_requests(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE mkt.return_requests ADD CONSTRAINT fk_returns_refund FOREIGN KEY (refund_id) REFERENCES mkt.purchase_orders(id) DEFERRABLE INITIALLY DEFERRED;

ALTER TABLE fin.payment_intents ADD CONSTRAINT fk_intents_insurance FOREIGN KEY (insurance_id) REFERENCES fin.refunds(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE fin.refunds         ADD CONSTRAINT fk_refunds_cancellation FOREIGN KEY (cancellation_id) REFERENCES fin.refunds(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE fin.refunds         ADD CONSTRAINT fk_refunds_case FOREIGN KEY (case_id) REFERENCES fin.settlements(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE fin.settlements     ADD CONSTRAINT fk_settlements_payout FOREIGN KEY (payout_id) REFERENCES fin.settlement_adjustments(id) DEFERRABLE INITIALLY DEFERRED;
ALTER TABLE fin.settlement_adjustments ADD CONSTRAINT fk_adjustments_case FOREIGN KEY (case_id) REFERENCES fin.settlements(id) DEFERRABLE INITIALLY DEFERRED;

ALTER TABLE ops.cases           ADD CONSTRAINT fk_cases_conversation FOREIGN KEY (conversation_id) REFERENCES ops.cases(id) DEFERRABLE INITIALLY DEFERRED;

```

22. Triggers, append-only tables and invariants

```
-- 1) updated_at on every table that has the column
DO $$
DECLARE r record;
BEGIN
  FOR r IN
    SELECT c.table_schema, c.table_name
      FROM information_schema.columns c
    JOIN information_schema.tables t
      ON t.table_schema = c.table_schema AND t.table_name = c.table_name AND t.table_type = 'I'
     WHERE c.column_name = 'updated_at'
      AND c.table_schema IN ('identity', 'ref', 'org', 'files', 'svc', 'freight', 'transport',
                             'customs', 'mkt', 'fin', 'ops', 'msg', 'sys', 'cms')

  LOOP
    EXECUTE format('CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON %I.%I
                    FOR EACH ROW EXECUTE FUNCTION core.set_updated_at()', r.table_schema, r.tabl

  END LOOP;
END $$;
```

```
-- 2) Append-only tables: triggers + REVOKE (§24)
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY[
    'org.consent', 'files.document_versions', 'files.document_reviews', 'svc.order_events', 'sv
    'transport.trip_events', 'warehouse.stock_movements', 'fin.ledger_transactions', 'fin.ledger
    'fin.invoice_lines', 'ops.case_internal_notes', 'audit.audit_logs']
  LOOP
    EXECUTE format('CREATE TRIGGER trg_append_only BEFORE UPDATE OR DELETE ON %s
                    FOR EACH ROW EXECUTE FUNCTION core.forbid_mutation()', t);

  END LOOP;
END $$;
```

```
-- 3) Double-entry invariant: debits = credits per ledger transaction, checked at commit
CREATE OR REPLACE FUNCTION fin.assert_transaction_balanced() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  v_debits bigint;
  v_credits bigint;
BEGIN
  SELECT coalesce(sum(amount) FILTER (WHERE direction = 'DEBIT'), 0),
```

```

        coalesce(sum(amount) FILTER (WHERE direction = 'CREDIT'), 0)
    INTO v_debits, v_credits
    FROM fin.ledger_entries
    WHERE transaction_id = NEW.transaction_id;
    IF v_debits = 0 OR v_debits <> v_credits THEN
        RAISE EXCEPTION 'Ledger transaction % is unbalanced (debits %, credits %)', NEW.transaction_
    END IF;
    RETURN NULL;
END $$;

```

```

CREATE CONSTRAINT TRIGGER trg_ledger_balanced
    AFTER INSERT ON fin.ledger_entries
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION fin.assert_transaction_balanced();

```

```

-- 4) Issued invoices are immutable (only ZATCA fields and file links may change); no deletes
CREATE OR REPLACE FUNCTION fin.protect_invoice() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF (NEW.number, NEW.invoice_type, NEW.issuer_org_id, NEW.recipient_org_id, NEW.recipient_name,
        NEW.issue_at, NEW.subtotal, NEW.vat_amount, NEW.total)
        IS DISTINCT FROM
        (OLD.number, OLD.invoice_type, OLD.issuer_org_id, OLD.recipient_org_id, OLD.recipient_name,
        OLD.issue_at, OLD.subtotal, OLD.vat_amount, OLD.total) THEN
        RAISE EXCEPTION 'Issued invoice % is immutable; issue a credit note instead', OLD.number;
    END IF;
    RETURN NEW;
END $$;

```

```

CREATE TRIGGER trg_invoices_immutable BEFORE UPDATE ON fin.invoices FOR EACH ROW EXECUTE FUNCTION
CREATE TRIGGER trg_invoices_no_delete BEFORE DELETE ON fin.invoices FOR EACH ROW EXECUTE FUNCTION

```

```

-- 5) Marketplace search vector (category and supplier names are appended by the search indexing
ALTER TABLE mkt.products ADD CONSTRAINT ck_products_specs_array CHECK (jsonb_typeof(specs) = 'array')

```

```

CREATE OR REPLACE FUNCTION mkt.products_search_vector() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.search_vector :=
        setweight(to_tsvector('simple', core.normalize_ar(NEW.name)), 'A')
    || setweight(to_tsvector('simple', core.normalize_ar(coalesce(
        (SELECT string_agg(e->>'value', ' ') FROM jsonb_array_elements(NEW.specs) AS e), ''))),
        'C');
    RETURN NEW;

```

```

END $$;
CREATE TRIGGER trg_products_search
  BEFORE INSERT OR UPDATE OF name, description, specs ON mkt.products
  FOR EACH ROW EXECUTE FUNCTION mkt.products_search_vector();

```

Invariants: where each one is enforced

Invariant	Database	Application
Money totals add up (quotes, orders, POs, checkout, invoices, settlements, charges)	CHECK constraints	Computes the values
Ledger balanced per transaction	Deferred constraint trigger	Builds postings
One live quote per provider/request, one live payment intent per payable, one open cancellation per order, one live broker authorization per order	Partial unique indexes	Friendly 409 errors
Stock never negative	CHECK ON inventory_items	Row lock + movement insert in one transaction; nightly Σ check
Marketplace reservations $\leq$ stock	CHECK ON products	Reservation TTL job
Commission rules don't overlap in time	EXCLUDE constraint	Maker-checker flow
Maker $\neq$ checker	CHECK on batches, approvals, rules, refunds, cancellations	Hides approve button
State transitions (request, quote, order, payment, settlement, document, KYB, return, case, ...)	Enum values only	Transition tables (04-state-machines.md)
Tenant isolation (org scoping)	FKs only	Every query scoped by the membership org (RLS optional later)

Invariant	Database	Application
Refunds $\leq$ amount paid	refunded_amount <= amount	Sums refunds before creating one
One active trip per driver / vehicle	—	Row lock on driver and vehicle at assignment
Evidence file ownership and scan status	—	Validated on attach (arrays have no FK)
Append-only history	Triggers + REVOKE	Correction rows instead of updates

23. Partitioning and retention

Partitioned tables use monthly `RANGE` partitions managed by **pg_partman** (3 months pre-created plus a default partition).

```
-- Example for one table; repeat for msg.notifications (created_at) and audit.audit_logs (occurred_at)
SELECT partman.create_parent(
    p_parent_table => 'transport.tracking_points',
    p_control       => 'recorded_at',
    p_interval      => '1 month',
    p_premake       => 3
);
-- Without pg_partman:
-- CREATE TABLE transport.tracking_points_2026_10 PARTITION OF transport.tracking_points
--     FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

Data	Retention (proposal, confirm with legal / tax advisor)	Mechanism
transport.tracking_points	Full detail 7 days → downsampled to 1 point / 60 s → dropped after 12 months	Downsampling job + partition drop
msg.notifications	12 months	Partition drop

Data	Retention (proposal, confirm with legal / tax advisor)	Mechanism
audit.audit_logs	24 months hot; finance-related actions archived to cold storage (Parquet on S3) for 7 years	Archive job then partition drop
identity.otp_challenges	30 days	Daily delete
sys.outbox_events	7 days after publish	Daily delete
sys.processed_events	30 days	Daily delete
sys.idempotency_keys	24 h (expires_at)	Hourly delete
Draft svc.service_requests	30 days after last update	Daily purge (details cascade)
Unattached files.files	7 days	Daily delete (object + row)
Chat messages and attachments	2 years after the conversation closes	Monthly job
Orders, documents, finance (svc , files.documents , fin)	At least as long as tax and commercial record-keeping rules require. Plan for 10 years	Never purged by jobs; archived later if needed
Personal data of deleted accounts	Anonymized on deletion (PDPL); financial records kept without personal fields where the law allows	Anonymization job

24. Reporting views and grants

```
-- Application role
GRANT USAGE ON SCHEMA core, identity, ref, org, files, svc, freight, transport, warehouse, customs,
      mkt, fin, ops, msg, audit, sys, cms TO logix_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA
      identity, ref, org, files, svc, freight, transport, warehouse, customs, mkt, fin, ops, msg
      TO logix_app;
GRANT SELECT, INSERT ON ALL TABLES IN SCHEMA audit TO logix_app;
REVOKE UPDATE, DELETE ON org.consent, files.document_versions, files.document_reviews, svc.orders,
      svc.quote_revisions, transport.trip_events, warehouse.stock_movements, fin.ledger_transactions,
      fin.ledger_entries, fin.invoice_lines, ops.case_internal_notes FROM logix_app;
REVOKE DELETE ON fin.invoices FROM logix_app;
ALTER DEFAULT PRIVILEGES FOR ROLE logix_migrator IN SCHEMA svc, mkt, fin, ops, msg, sys
      GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO logix_app;

-- Reporting role (Metabase / read replica): masked views only
GRANT USAGE ON SCHEMA reporting TO logix_reporting;

CREATE VIEW reporting.v_orders AS
SELECT o.id, o.reference, o.service_type, o.status, o.payment_status, o.settlement_status,
      o.customer_org_id, o.provider_org_id, o.total_amount, o.vat_amount, o.commission_rate_bp,
      o.commission_amount, o.commission_vat_amount, o.net_to_provider,
      r.origin_city_id, r.destination_city_id, r.origin_country_code, r.destination_country_code,
      o.created_at, o.confirmed_at, o.pod_at, o.completed_at, o.cancelled_at
FROM svc.orders o
JOIN svc.service_requests r ON r.id = o.request_id;

CREATE VIEW reporting.v_quotes AS -- never exposes internal_cost / target_margin
SELECT q.id, q.request_id, q.provider_org_id, q.status, q.total_amount, q.service_fee, q.vat_amount,
      q.valid_until, q.submitted_at, q.accepted_at
FROM svc.quotes q;

CREATE VIEW reporting.v_organizations AS -- individuals are anonymized
SELECT o.id, o.kind,
      CASE WHEN o.kind = 'INDIVIDUAL' THEN 'Individual customer' ELSE o.display_name END AS display_name,
      o.status, o.rating_avg, o.rating_count, o.completed_jobs_count, bp.verification_status, bp.verified_at
FROM org.organizations o
LEFT JOIN org.business_profiles bp ON bp.organization_id = o.id
LEFT JOIN org.addresses a ON a.id = bp.registered_address_id;

GRANT SELECT ON ALL TABLES IN SCHEMA reporting TO logix_reporting;
```

Further views follow the same pattern (no phones, emails, IBANs, national IDs, document keys or provider-private fields): `v_requests` , `v_payments` , `v_refunds` , `v_settlements` , `v_payouts` , `v_purchase_orders` , `v_products` , `v_returns` , `v_cases` , `v_kyb_cases` , `v_licenses_expiry` , `v_insurance` .

25. Prisma mapping notes

Applies if T-02 confirms Prisma.

Topic	Approach
Multiple schemas	<code>datasource db { schemas = ["core", "identity", "ref", ...] }</code> with the <code>multiSchema</code> feature (a preview flag on older Prisma versions); every model and enum gets <code>@@schema("...")</code>
Naming	Models in PascalCase with <code>@@map("service_requests")</code> ; fields in camelCase with <code>@map("snake_case")</code>
IDs	<code>String @id @db.Uuid</code> , generated in the app with a UUIDv7 library. Don't use <code>@default(uuid())</code> , which generates v4
Money	<code>BigInt</code> in Prisma → JS <code>bigint</code> . Convert at the API boundary with a Prisma client extension. SAR amounts in halalas stay below 2^{53} , so <code>Number()</code> is safe
Domains	Prisma sees the base type (<code>bigint</code> , <code>integer</code>). The domain <code>CHECK</code> s live in SQL migrations
PostGIS	<code>Unsupported("geography(Point, 4326)")</code> ; read and write with <code>\$queryRaw / TypedSQL</code>
<code>citext</code> , <code>inet</code> , <code>cidr[]</code>	<code>@db.Citext</code> , <code>@db.Inet</code> ; <code>cidr[]</code> as <code>Unsupported</code>
Not expressible in the Prisma schema	Partial unique indexes, expression/GIN indexes, <code>CHECK</code> and <code>EXCLUDE</code> constraints, generated columns, triggers, partitioned tables, views, grants. Write them in SQL migrations (<code>prisma migrate dev --create-only</code> , then edit) and check <code>prisma migrate diff</code> in CI so they are never dropped silently
Partitioned tables	Queried like normal tables. Partitions are created by migrations / <code>pg_partman</code>

Topic	Approach
Enum arrays	Supported (<code>CustomsMovement[]</code>)
Alternative	If Drizzle is chosen instead, <code>pgSchema</code> , enums, partial indexes and checks are expressible in TypeScript, so less raw SQL is needed. This is part of the T-02 tradeoff

26. Seed data

Table	Seed
<code>ref.countries</code> , <code>ref.regions</code> , <code>ref.cities</code>	ISO countries; Saudi regions and major cities; GCC and main trade-partner cities
<code>ref.ports</code>	Saudi seaports (e.g. Jeddah Islamic Port <code>SAJED</code> , Dammam <code>SADMM</code>), airports (<code>RUH</code> , <code>JED</code> , <code>DMM</code>), land borders and dry ports (internal codes), main foreign ports used in demos (<code>CNSHA</code> , <code>AEJEA</code> , <code>DXB</code>). Codes are verified against UN/LOCODE and IATA lists when loading
<code>ref.customs_checkpoints</code>	One per customs-enabled port / border / dry port
<code>ref.vehicle_types</code> , <code>ref.container_types</code> , <code>ref.storage_types</code> , <code>ref.handling_services</code> , <code>ref.units_of_measure</code> , <code>ref.commodity_categories</code>	The values listed in modules/03-reference-data.md
<code>ref.document_types</code> , <code>ref.document_requirements</code>	The document table and required-document matrix from modules 02, 03 and 10
<code>ref.reason_codes</code>	Cancellation, return, trip issue, KYB and document-review reasons with ar/en user messages
<code>ref.holidays</code>	Current and next year: National Day (23 Sep), Founding Day (22 Feb), Eid holidays (entered when announced)

Table	Seed
identity.staff_roles , identity.staff_role_permissions	Built-in roles and the matrix in web_dashboard/md/03-roles-permissions.md . The first super admin is created by a one-off bootstrap command, not a seed
fin.commission_rules	FREIGHT_TRANSPORT 1000 bps, CUSTOMS_STORAGE 2000 bps, MARKETPLACE 350 bps (supplied rates; base and VAT per D-02)
fin.ledger_accounts	Platform-level accounts (gateway clearing, customer payments held, commission revenue, VAT output payable, refunds payable, cancellation fees, payouts in transit, platform bank). Per-organization payables are created on first use
ops.cancellation_rules	The D-01 proposal, including REQUIRES_REVIEW rows for the overlap and for warehousing
org.terms_documents	Draft v0 per audience and locale (unpublished until legal sign-off)
msg.notification_templates	The catalog in modules/15-messaging-notifications.md , in ar and en
sys.app_config	calendar.weekend = ["FRI","SAT"] , upload.maxSizeMb = 10 , request.validityDays = 7 , quote.defaultValidityHours = 48 , payment.holdMinutes = 30 , settlement.businessDays = 3 , returns.windowDays = 3 , damageReport.windowHours = 24 , autoAcceptance = {enabled: false, hours: 72} (D-10), thresholds.refundMakerChecker = 500000 (halalas)
sys.feature_flags	service.shipping/transport/warehousing/customs.enabled (D-25), marketplace.checkout.enabled , marketplace.premoderation , payments.applePay , insurance.enabled , businessFinance.enabled = false
Demo data (non-production only)	One organization per role, and sample requests and orders per service mirroring the client PDFs (LX-2048, WH-105, PO-1048, RT-1048, CASE-2048)